Computer Organization & Architecture

Complete GTU Notes — All 11 Units

Detailed Answers · ASCII Diagrams · Algorithms · Numerical Examples

GTU EXAM READY | 100% Syllabus Covered

- Unit 1: Structure of Computers (2 hrs, 5%)
- Unit 2: Register Transfer and Micro-operations (6 hrs, 10%)
- Unit 3: Basic Computer Organization and Design (4 hrs, 15%) ★
- Unit 4: Assembly Language Programming (6 hrs, 10%)
- Unit 5: Micro-programmed Control Organization (2 hrs, 5%)
- Unit 6: Central Processing Unit (6 hrs, 15%) ★
- Unit 7: Pipeline and Vector Processing (5 hrs, 10%)
- Unit 8: Input-Output Organization (4 hrs, 10%)
- Unit 9: Memory Organization (4 hrs, 10%) ★
- Unit 10: Computer Arithmetic (4 hrs, 10%)
- Unit 11: Multiprocessors (4 hrs, 5%)

Unit 1: Structure of Computers

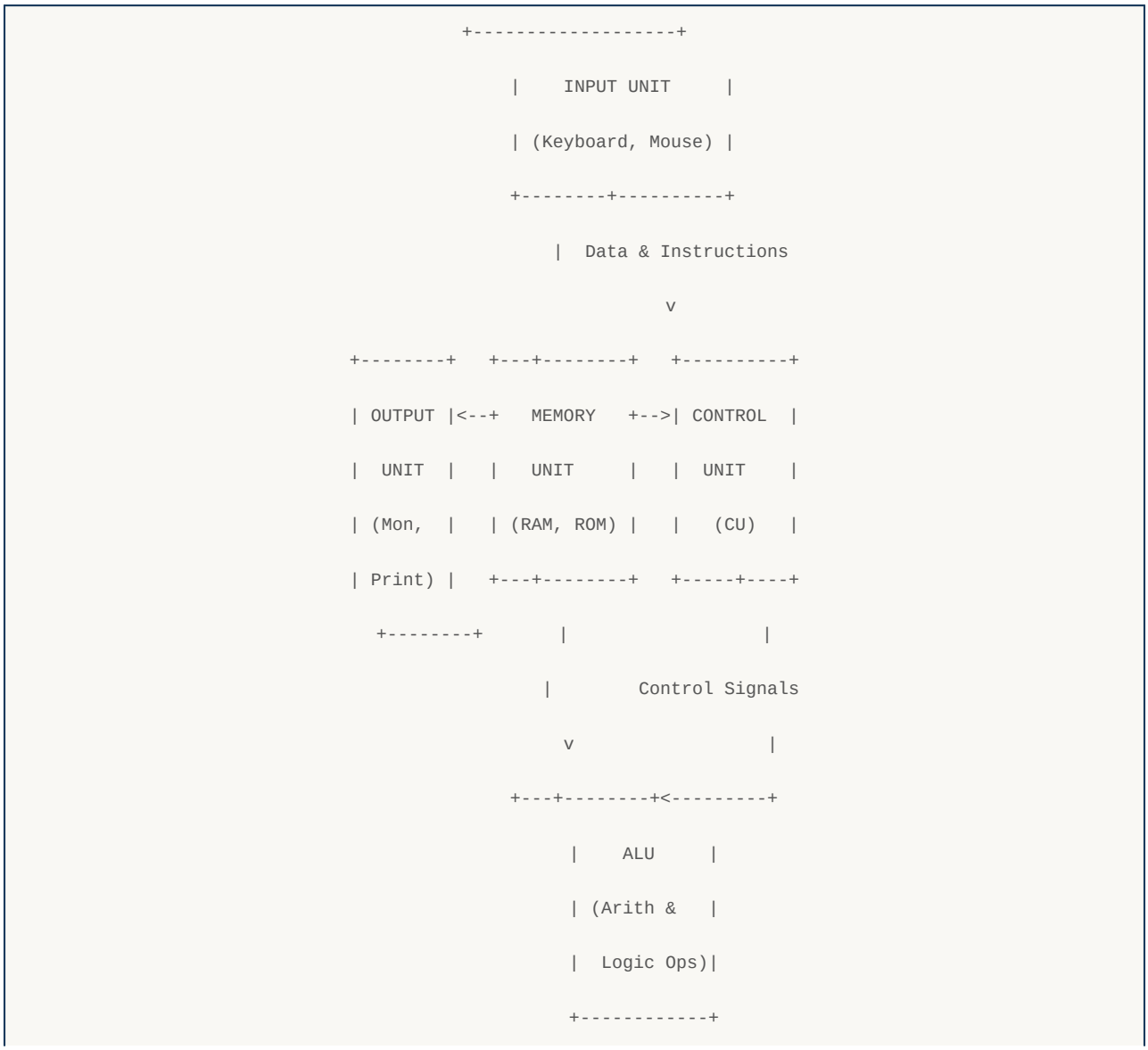
2 Hours 5%

Q1. Explain Basic Functional Units of a Digital Computer with Neat Diagram

A digital computer is an electronic device that processes data represented in binary form. It consists of five basic functional units that work together to perform computations.

Five Basic Functional Units:

- **Input Unit:** Accepts data and instructions from the user via keyboard, mouse, scanner, etc. Converts external data into internal binary form understandable by the computer.
- **Output Unit:** Presents processed results to the user via monitor, printer, speaker. Converts internal binary results back to human-understandable form.
- **Memory Unit (Storage):** Stores data and programs temporarily (RAM) or permanently (ROM, HDD). Has two levels: Primary Memory (fast, volatile) and Secondary Memory (slow, non-volatile).
- **Arithmetic and Logic Unit (ALU):** Performs all arithmetic operations (add, subtract, multiply, divide) and logical operations (AND, OR, NOT, XOR). This is where actual computation happens.
- **Control Unit (CU):** Directs and coordinates all other units. Fetches instructions from memory, decodes them, and issues control signals. Acts as the 'brain' of the computer.



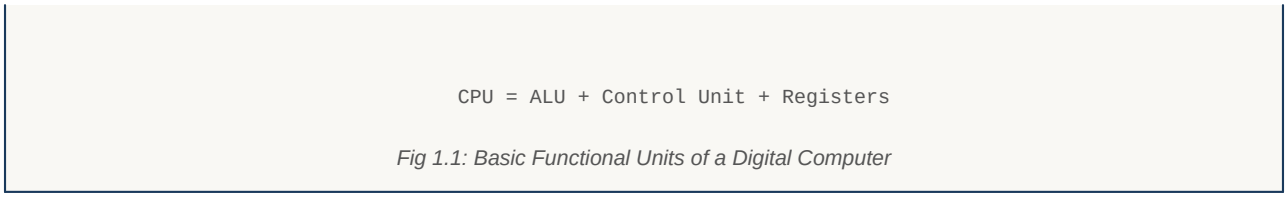


Fig 1.1: Basic Functional Units of a Digital Computer

Q2. Explain Von Neumann Architecture. Compare with Harvard Architecture.

Von Neumann Architecture:

Proposed by John Von Neumann in 1945. The key concept is the **Stored Program Concept** — both data and program instructions are stored in the same memory using the same bus.

- Single shared memory for both instructions and data.
- Single bus (data bus) connects CPU and memory.
- Instructions are fetched and data is accessed sequentially (one at a time).
- Simple design — widely used in modern computers.
- **Bottleneck:** CPU and memory share one bus, creating 'Von Neumann Bottleneck'.

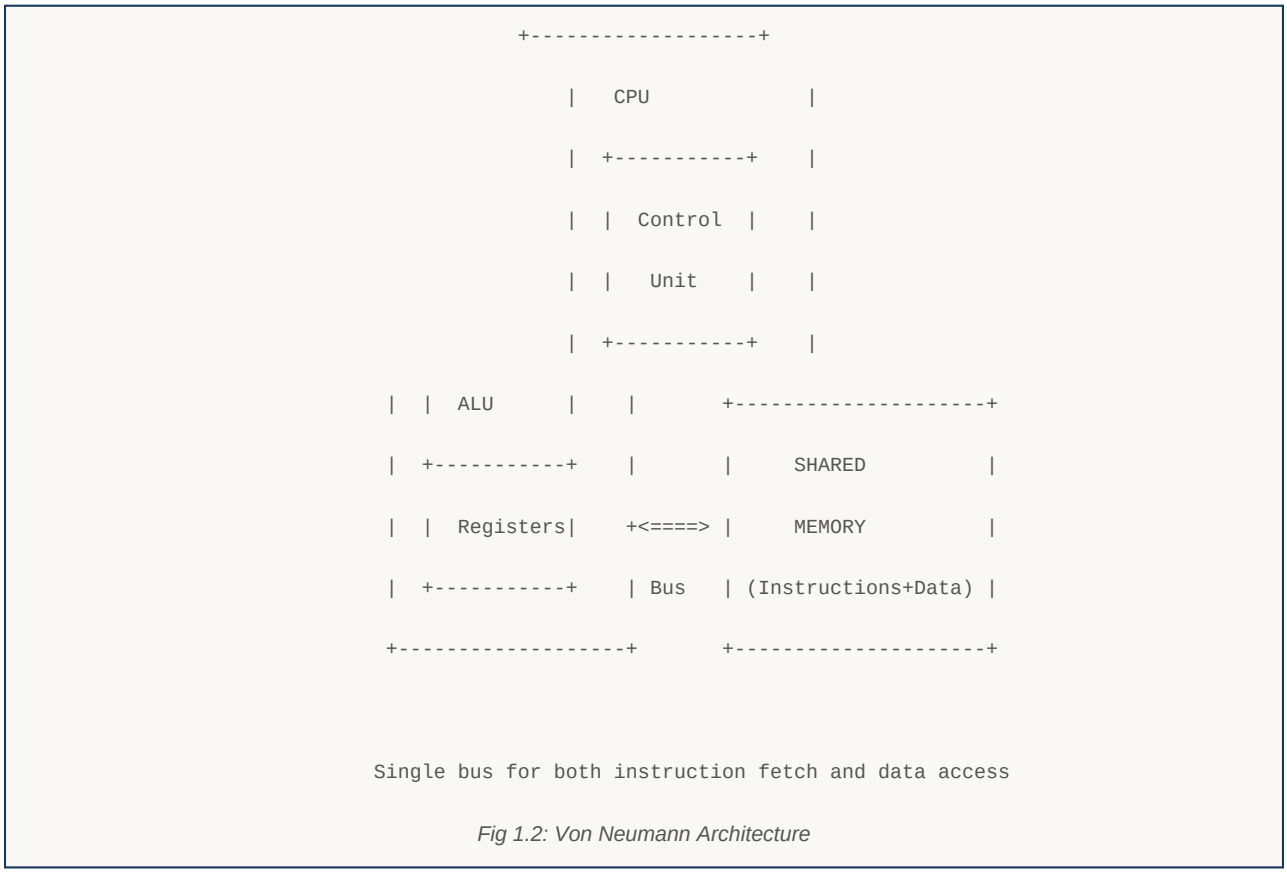


Fig 1.2: Von Neumann Architecture

Harvard Architecture:

Uses **separate memories and buses** for instructions and data. Allows simultaneous fetching of instruction and data — no bottleneck.

- Separate instruction memory and data memory.
- Two independent buses — faster operation.
- CPU can fetch next instruction while reading/writing data.
- Used in DSP processors, microcontrollers (PIC, AVR), modern CPUs (cache level).



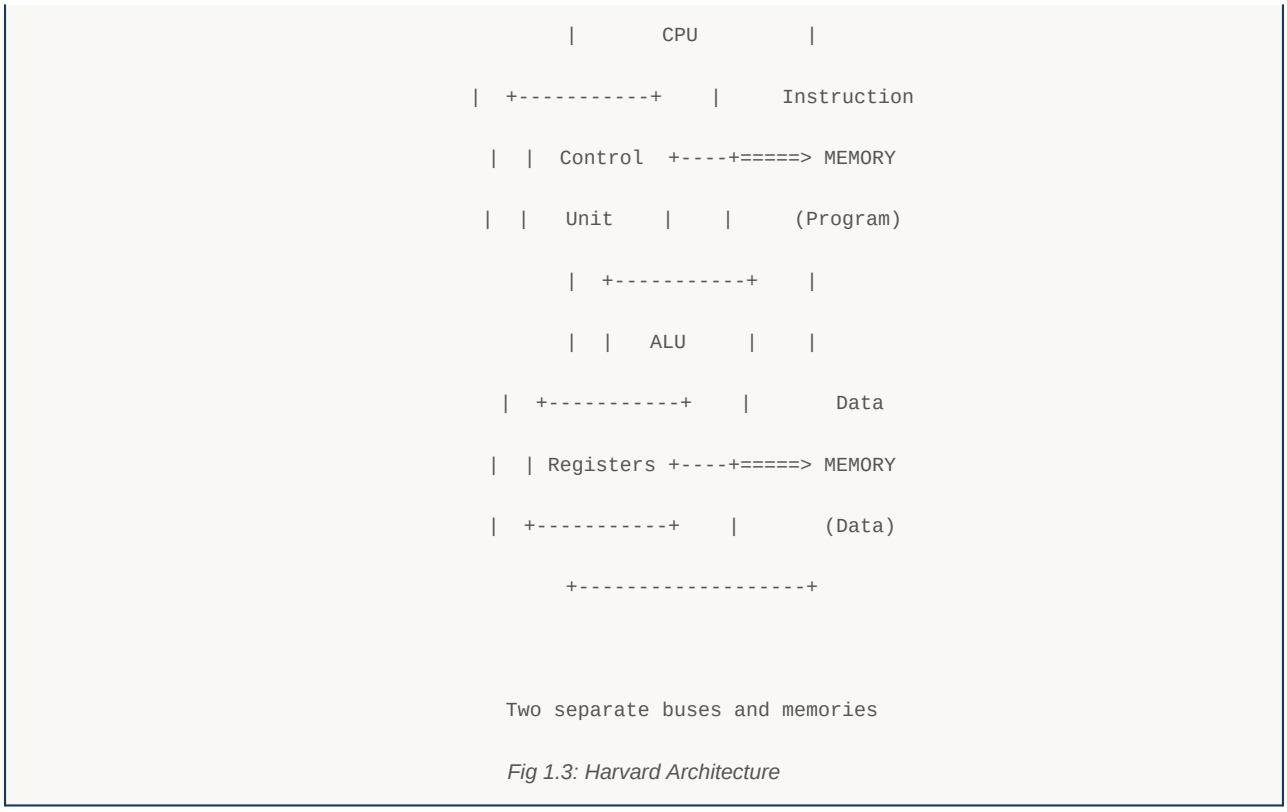


Fig 1.3: Harvard Architecture

Feature	Von Neumann	Harvard
Memory	Single shared memory	Separate instruction & data memory
Bus	Single bus (shared)	Two separate buses
Speed	Slower (bottleneck)	Faster (parallel access)
Cost	Less expensive	More expensive
Flexibility	More flexible	Less flexible
Used in	General-purpose computers	DSP, microcontrollers, modern CPUs
Bottleneck	Yes (Von Neumann bottleneck)	No bottleneck

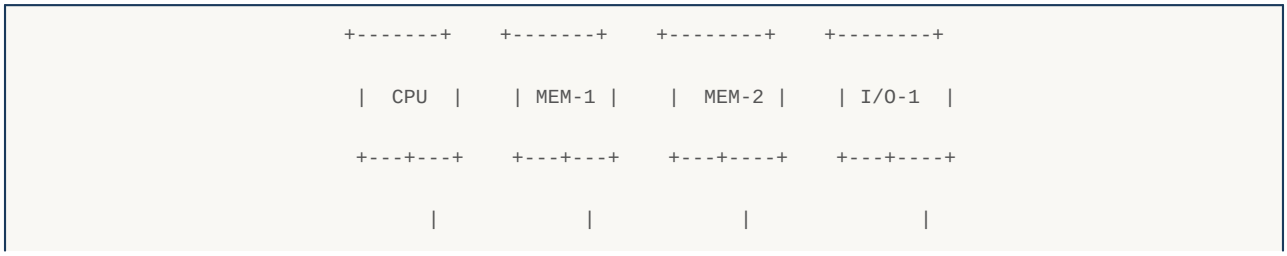
Q3. Explain Bus Structures (Single Bus, Multi-Bus) with Diagram

What is a Bus?

A bus is a collection of wires (electrical paths) that transfer data between components of a computer. It consists of: **Data Bus** (carries data), **Address Bus** (carries memory addresses), and **Control Bus** (carries control signals).

Single Bus Structure:

All components (CPU, Memory, I/O devices) are connected to a single common bus. Simple and economical but only one device can use the bus at a time — creates bottleneck.



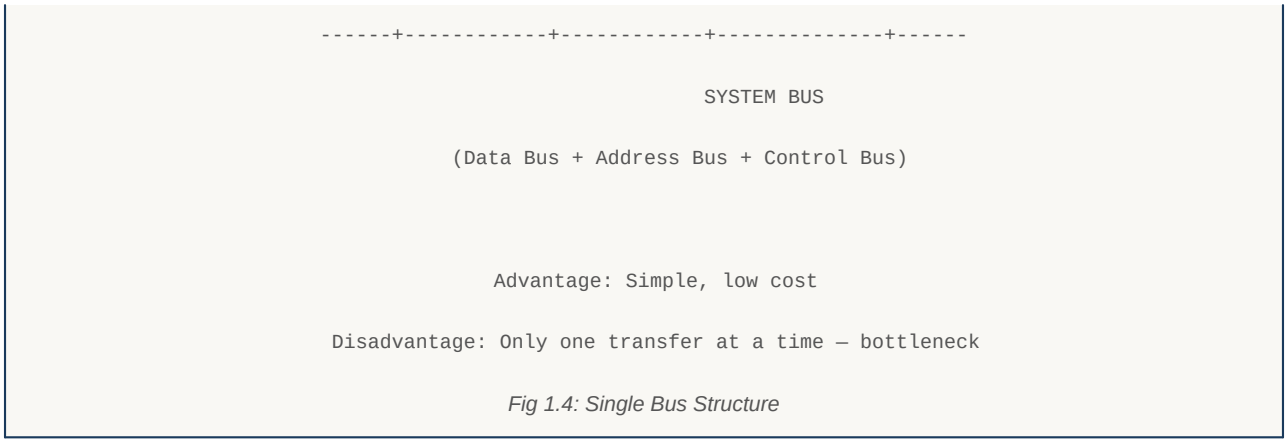


Fig 1.4: Single Bus Structure

Multiple Bus Structure:

Uses separate buses for different purposes. Reduces bottleneck. Allows simultaneous transfers on different buses.

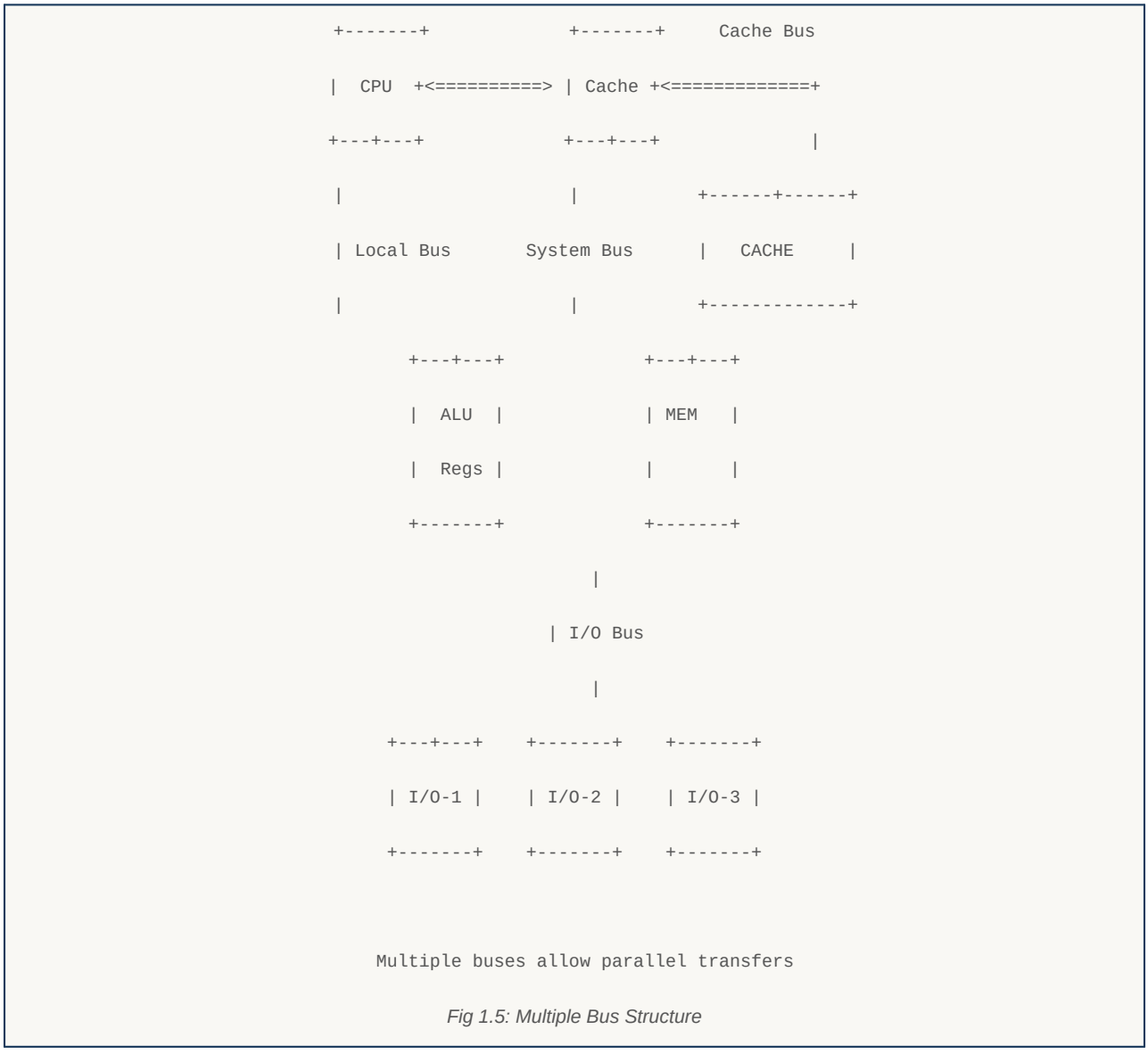


Fig 1.5: Multiple Bus Structure

Q4. Data Representation: Fixed Point and Floating Point (IEEE 754)

Fixed Point Representation:

In fixed-point representation, the binary point (decimal point equivalent) is assumed to be at a fixed position. Used for integers and simple fractions.

- Integers: binary point assumed after the rightmost bit. Example: 01101 = 13.
- Fractions: binary point assumed before the leftmost bit. Example: .1101 = 0.5+0.25+0.0625 = 0.8125.
- Sign-Magnitude: MSB is sign bit (0=+, 1=–). Example: 10001101 = –13.
- 1's Complement: Invert all bits for negative. Example: –13 = 11110010.
- 2's Complement: Most common. Add 1 to 1's complement. Example: –13 = 11110011.

Floating Point Representation (IEEE 754):

Floating-point represents numbers in scientific notation: **N = M × 2^E**, where M = Mantissa (significand) and E = Exponent.

IEEE 754 Single Precision (32-bit):

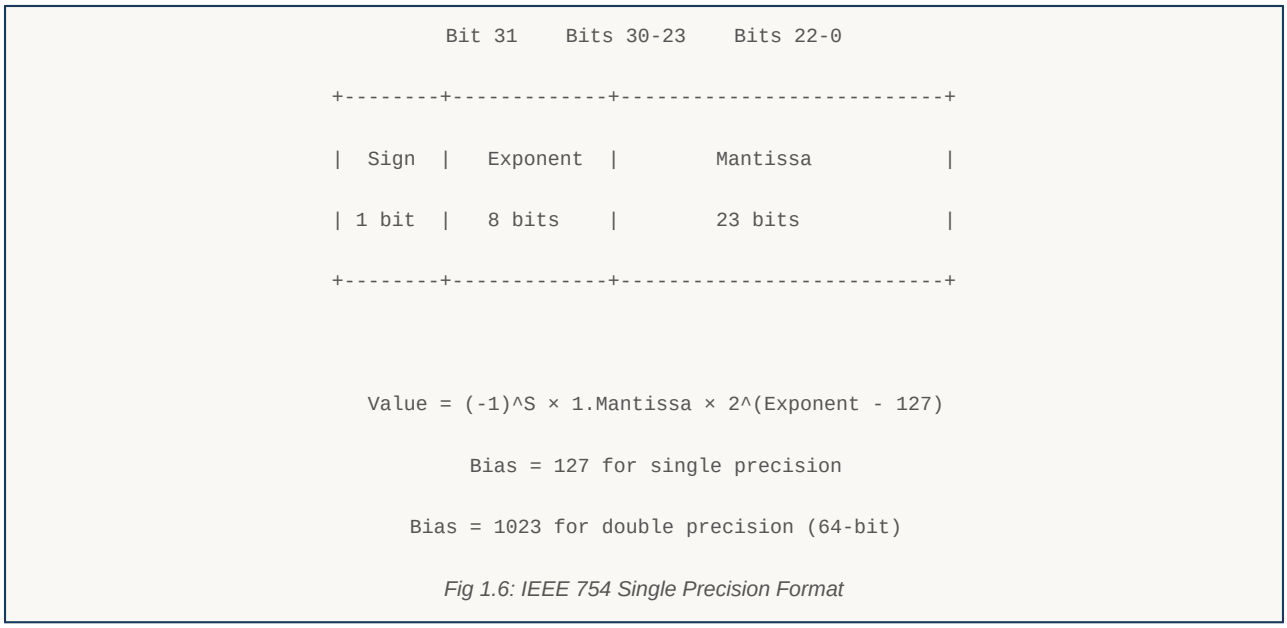


Fig 1.6: IEEE 754 Single Precision Format

Example: Represent –6.5 in IEEE 754 Single Precision

```
Step 1: Sign bit = 1 (negative number)
Step 2: Convert 6.5 to binary = 110.1
Step 3: Normalize: 1.101 × 2^2
Step 4: Exponent = 2 + 127 = 129 = 10000001 (binary)
Step 5: Mantissa = 101 followed by 20 zeros = 10100000000000000000000

Result:
Sign | Exponent  | Mantissa
  1  | 10000001  | 10100000000000000000000

Full 32-bit: 1 10000001 10100000000000000000000
```

Q5. Error Detection and Correction Codes (Hamming Code, Parity)

Parity Bit:

A parity bit is added to a data word to make the total number of 1s either even (even parity) or odd (odd parity). Detects single-bit errors but cannot correct them.

Example:

```
Data: 1011
Even parity: count of 1s = 3 (odd) → add parity bit 1 → 10111
```

```
Even parity: count of 1s = 4 (even) → Transmitted: 10111
If received: 10011 → count of 1s = 3 → Error detected!
Cannot determine which bit is wrong → cannot correct.
```

Hamming Code:

Hamming code can detect and **correct single-bit errors**. It adds redundant (parity) bits at positions that are powers of 2 (1, 2, 4, 8, ...).

Rules for Hamming Code:

- Parity bits are placed at positions 2^0=1, 2^1=2, 2^2=4, 2^3=8, ...
- Each parity bit covers specific bit positions based on binary representation.
- Total bits needed: If data has m bits, find r such that 2^r >= m+r+1.

Example: Encode data 1011 using Even Parity Hamming Code

```
Data bits: d1=1, d2=0, d3=1, d4=1
Need r parity bits: 2^r >= 4+r+1 → r=3 (since 2^3=8 >= 8) ✓

Bit positions: 1  2  3  4  5  6  7
               P1 P2 d1 P4 d2 d3 d4
               P1 P2  1 P4  0  1  1

Parity bit P1 covers positions 1,3,5,7: bits = 1,0,1 → P1=0 (even parity)
Parity bit P2 covers positions 2,3,6,7: bits = 1,1,1 → P2=1 (even parity)
Parity bit P4 covers positions 4,5,6,7: bits = 0,1,1 → P4=0 (even parity)

Final Hamming code: 0 1 1 0 0 1 1  (positions 1-7)

Error Correction: If bit position 5 flips to 1:
Received: 0 1 1 0 1 1 1
Recalculate parities → syndrome = 101 (binary) = 5
→ Error is in bit position 5 → flip it to correct.
```

Q6. Performance Parameters of Computers. Multiprocessor vs Multicomputer.

Performance Parameters:

- **Clock Speed (Hz):** Number of clock cycles per second. Higher = faster. Modern CPUs: 3-5 GHz.
- **MIPS (Million Instructions Per Second):** Measures instruction throughput.
- **MFLOPS:** Million Floating-Point Operations Per Second. Used for scientific computing.
- **CPI (Cycles Per Instruction):** Average clock cycles needed per instruction.
- **CPU Time = Instruction Count × CPI × Clock Cycle Time**
- **Throughput:** Total work done per unit time.
- **Latency (Response Time):** Time to complete a single task.
- **Bandwidth:** Rate of data transfer (bytes per second).

Performance Formula:

```
CPU Time = (Instruction Count × CPI) / Clock Rate
Speedup = Execution Time (old) / Execution Time (new)
MIPS = Clock Rate / (CPI × 10^6)
```

Feature	Multiprocessor	Multicomputer
Definition	Multiple CPUs sharing one common memory	Multiple complete computers connected by network

Memory	Shared memory (tightly coupled)	Distributed memory (loosely coupled)
Communication	Through shared memory (fast)	Through message passing over network (slower)
OS	Single operating system	Multiple OS instances
Examples	Multi-core processors, SMP	Cluster computing, Grid computing
Cost	Higher (shared memory expensive)	Lower (commodity hardware)
Scalability	Limited by memory bandwidth	Highly scalable
Coupling	Tightly coupled	Loosely coupled

Unit 2: Register Transfer and Micro-operations

6 Hours 10%

Q1. Explain Register Transfer Language (RTL) with Examples

Register Transfer Language (RTL) is a notation used to describe micro-operations — the elementary operations performed on data stored in registers. It specifies what data is transferred between registers and what operations are performed.

Basic RTL Notations:

Notation	Meaning	Example
R1 ← R2	Transfer contents of R2 to R1	R1 ← R2 (copy R2 into R1)
R1 ← R2 + R3	Add R2 and R3, store in R1	Arithmetic operation
R1 ← R2 ∧ R3	AND operation	Logical AND
R1 ← R2 ∨ R3	OR operation	Logical OR
R1 ← R2'	Complement of R2	NOT operation
R1(0-7) ← R2(0-7)	Transfer specific bits	Transfer lower 8 bits
If (P=1): R1 ← R2	Conditional transfer	Only if control signal P=1
M[AR] ← R1	Store R1 in memory at address AR	Memory write
R1 ← M[AR]	Load from memory	Memory read
PC ← PC + 1	Increment Program Counter	Common in fetch cycle

Example — Instruction Fetch using RTL:

```
Fetch Cycle RTL:
  MAR ← PC          ; Move Program Counter to Memory Address Register
  MBR ← M[MAR]      ; Read instruction from Memory into Memory Buffer Register
  PC  ← PC + 1      ; Increment PC to point to next instruction
  IR  ← MBR         ; Transfer instruction to Instruction Register

Execute ADD R1, R2:
  MAR ← R1          ; Address of operand 1
  MBR ← M[MAR]      ; Read operand 1
  A   ← MBR         ; Store in temp register A
  MAR ← R2          ; Address of operand 2
  MBR ← M[MAR]      ; Read operand 2
  B   ← MBR         ; Store in temp register B
  ALU_OUT ← A + B   ; ALU performs addition
  R1  ← ALU_OUT     ; Store result
```

Q2. Explain Bus and Memory Transfers (Three-State Bus Buffers)

Bus Transfer:

A bus is a shared pathway. To transfer data from one register to another via a bus, we need to select the source register, put its data on the bus, and enable the destination register to load from the bus.

```
Bus Transfer RTL:
  BUS ← R1    ; Select R1 as source (enable R1 output)
```

```
  R2 ← BUS    ; Load from bus into R2

For n registers sharing one bus:
  Only ONE register can drive the bus at a time
  Multiplexers select which register drives the bus
```

Three-State Bus Buffers:

Three-state (tri-state) buffers are electronic devices with three possible output states: Logic 0, Logic 1, and **High Impedance (Hi-Z)**. When in Hi-Z state, the buffer is effectively disconnected from the bus, allowing other devices to use it.

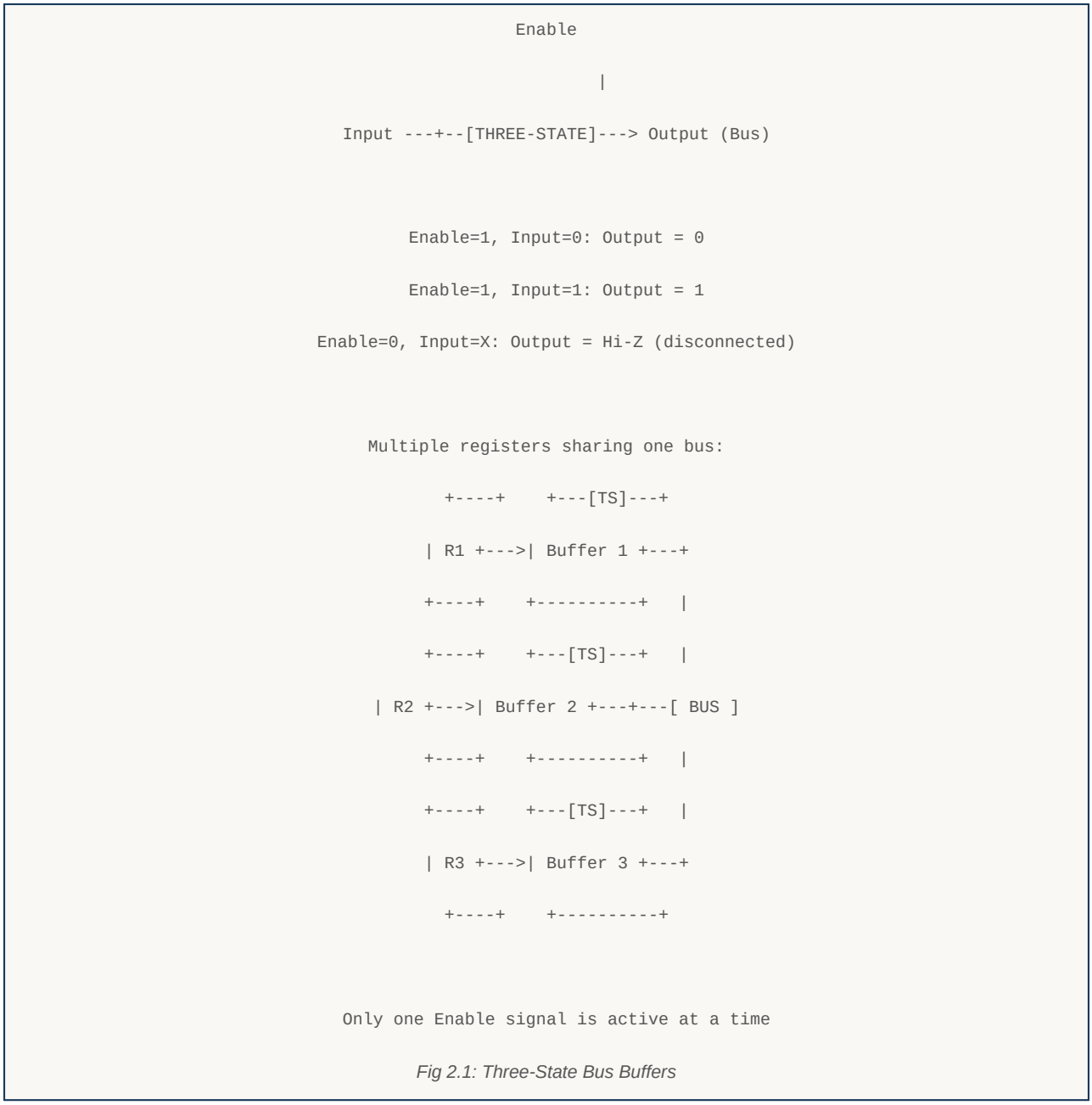


Fig 2.1: Three-State Bus Buffers

Memory Transfer RTL:

```
Read Operation:
  MAR ← Address        ; Load address into MAR
  MBR ← M[MAR]         ; Memory places data on bus → MBR
```

```
    R   ← MBR            ; Transfer to register

  Write Operation:
    MAR ← Address        ; Load address
    MBR ← R              ; Load data from register
    M[MAR] ← MBR         ; Memory stores MBR content
```

Q3. Arithmetic Micro-operations with Examples

Arithmetic micro-operations perform basic arithmetic operations on numeric data stored in registers.

Operation	RTL Notation	Description
Add	R1 ← R1 + R2	Add R1 and R2, store in R1
Subtract	R1 ← R1 - R2	Subtract R2 from R1
Increment	R1 ← R1 + 1	Add 1 to R1
Decrement	R1 ← R1 - 1	Subtract 1 from R1
Add with Carry	R1 ← R1 + R2 + Cin	Add with carry-in bit
Negate (2's comp)	R1 ← R1' + 1	2's complement of R1
Transfer	R1 ← R2	Copy R2 to R1

Binary Adder (Full Adder) — Core of ALU:

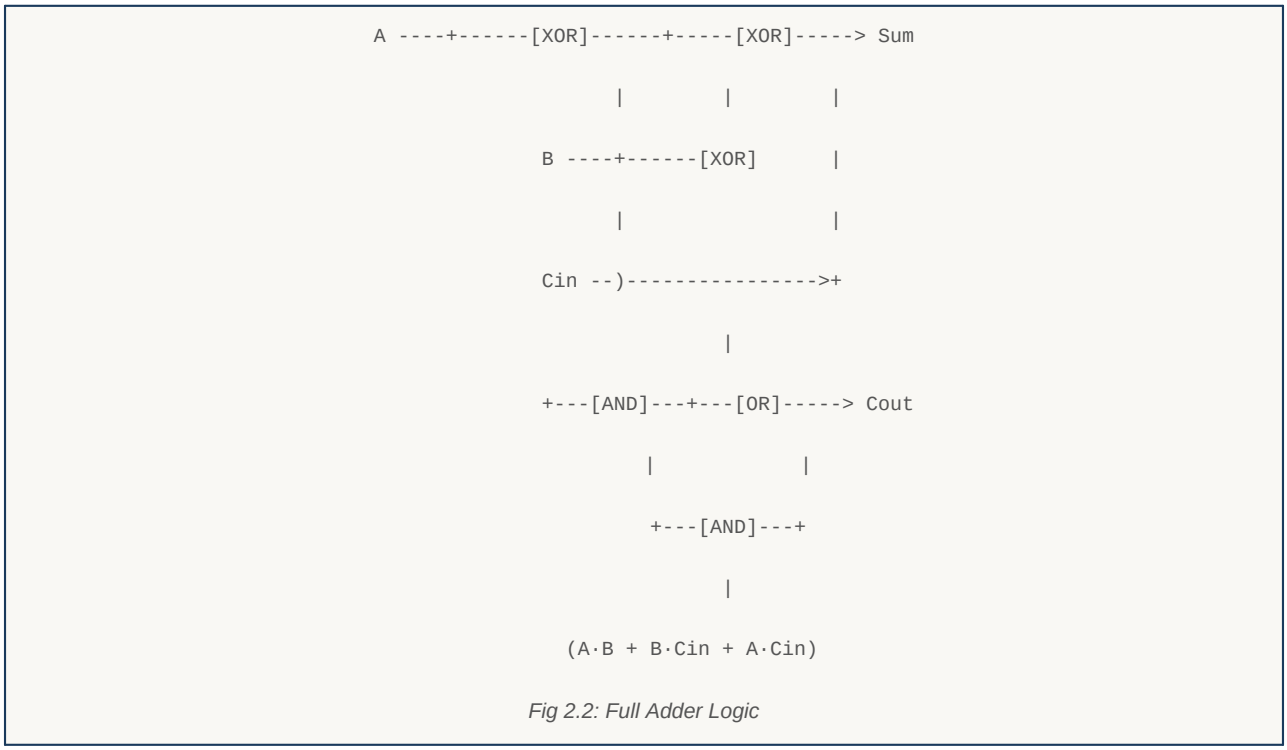


Fig 2.2: Full Adder Logic

Q4. Logic and Shift Micro-operations

Logic Micro-operations:

Perform bit-by-bit logical operations on register contents.

Operation	RTL	Example (A=1010, B=1100)
AND	R1 ← R1 ∧ R2	1010 AND 1100 = 1000

OR	R1 ← R1 ∨ R2	1010 OR 1100 = 1110
XOR	R1 ← R1 ⊕ R2	1010 XOR 1100 = 0110
NOT (Complement)	R1 ← R1'	NOT 1010 = 0101
NAND	R1 ← (R1 ∧ R2)'	NAND 1000 = 0111
NOR	R1 ← (R1 ∨ R2)'	NOR 1110 = 0001

Shift Micro-operations:

- **Logical Shift Left (shl):** Shift bits left, fill rightmost bit with 0. Equivalent to multiply by 2.
- **Logical Shift Right (shr):** Shift bits right, fill leftmost bit with 0. Equivalent to divide by 2.
- **Arithmetic Shift Left (shal):** Same as logical shift left (MSB/sign may change).
- **Arithmetic Shift Right (shar):** Shift right but preserve sign bit (MSB).
- **Rotate Left (rol):** Shift left; MSB wraps to LSB position.
- **Rotate Right (ror):** Shift right; LSB wraps to MSB position.

```
Example with 8-bit register: R = 1011 0110

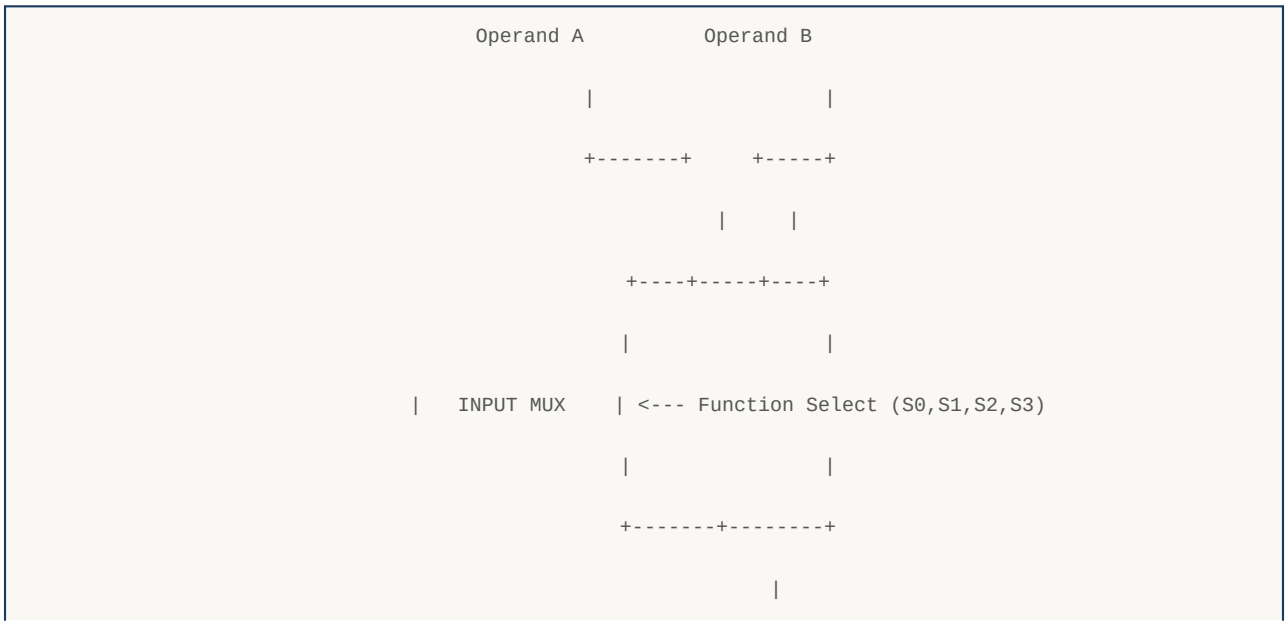
Logical Shift Left:  0110 1100  (0 inserted at right)
Logical Shift Right: 0101 1011  (0 inserted at left)
Arithmetic Right:    1101 1011  (sign bit 1 preserved)
Rotate Left:         0110 1101  (MSB 1 moved to LSB)
Rotate Right:        0101 1011  (LSB 0 moved to MSB)
```

Q5. Explain Arithmetic Logic Shift Unit (ALU) with Diagram

The **ALU (Arithmetic Logic Unit)** is the core computational component of the CPU. It performs all arithmetic, logical, and shift operations based on control signals from the Control Unit.

ALU Operations:

- Arithmetic: ADD, SUB, INC, DEC, ADD with carry
- Logical: AND, OR, XOR, NOT, NAND, NOR
- Shift: SHL, SHR, ROL, ROR, Arithmetic shifts
- Comparison: Compare two values, set flags



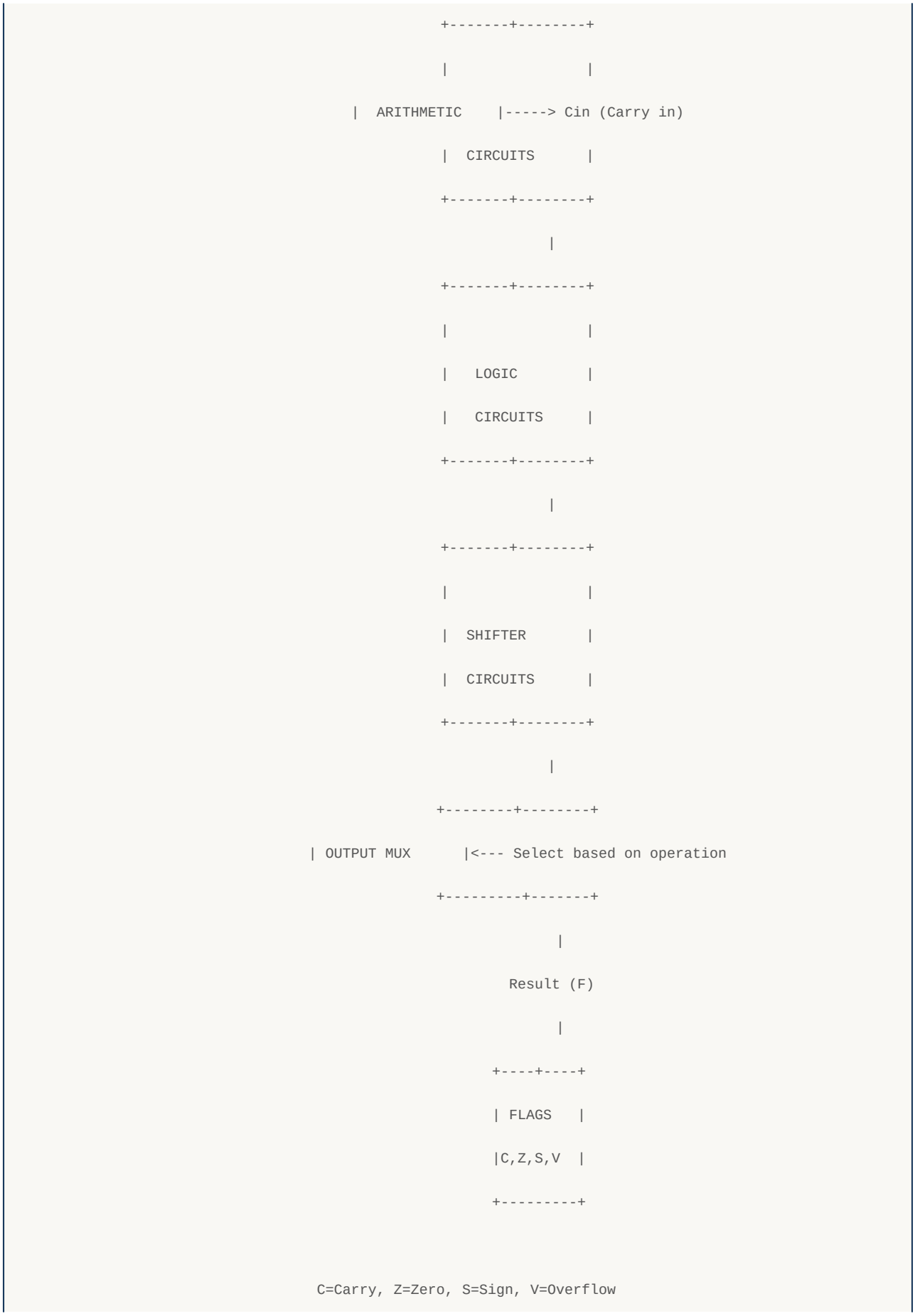

ARITHMETIC CIRCUITS
Cin (Carry in)
LOGIC CIRCUITS
SHIFTER CIRCUITS
OUTPUT MUX
Select based on operation
Result (F)
FLAGS
C,Z,S,V
C=Carry, Z=Zero, S=Sign, V=Overflow

Fig 2.3: Arithmetic Logic Shift Unit (ALU)

Unit 3: Basic Computer Organization and Design

4 Hours 15% ★

> **IMPORTANT:** This is the MOST IMPORTANT unit (15% marks). Master Instruction Cycle, Memory-Reference Instructions, and Design of Basic Computer.

Q1. Instruction Codes, Computer Registers, and Instruction Formats

Instruction Code:

An instruction code is a group of bits (binary pattern) that instructs the computer to perform a specific operation. An instruction typically has two parts:

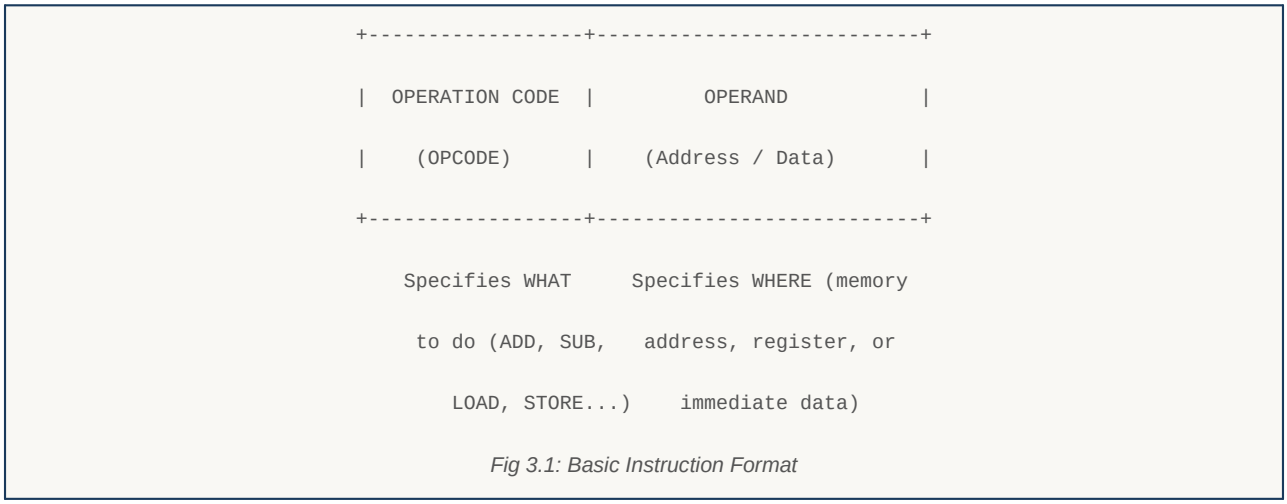


Fig 3.1: Basic Instruction Format

Computer Registers in Basic Computer:

Register	Full Name	Bits	Function
DR	Data Register	16	Holds operand fetched from memory
AR	Address Register	12	Holds memory address for read/write
AC	Accumulator	16	Main working register for ALU results
IR	Instruction Register	16	Holds current instruction being executed
PC	Program Counter	12	Holds address of next instruction
TR	Temporary Register	16	Holds temporary data during computation
INPR	Input Register	8	Holds input data from keyboard
OUTR	Output Register	8	Holds output data for display
SC	Sequence Counter	4	Counts timing pulses (T0-T15)
E	Extended AC / Carry	1	Extended bit of AC (carry flag)

Instruction Formats:

Memory-Reference Instruction (16-bit):



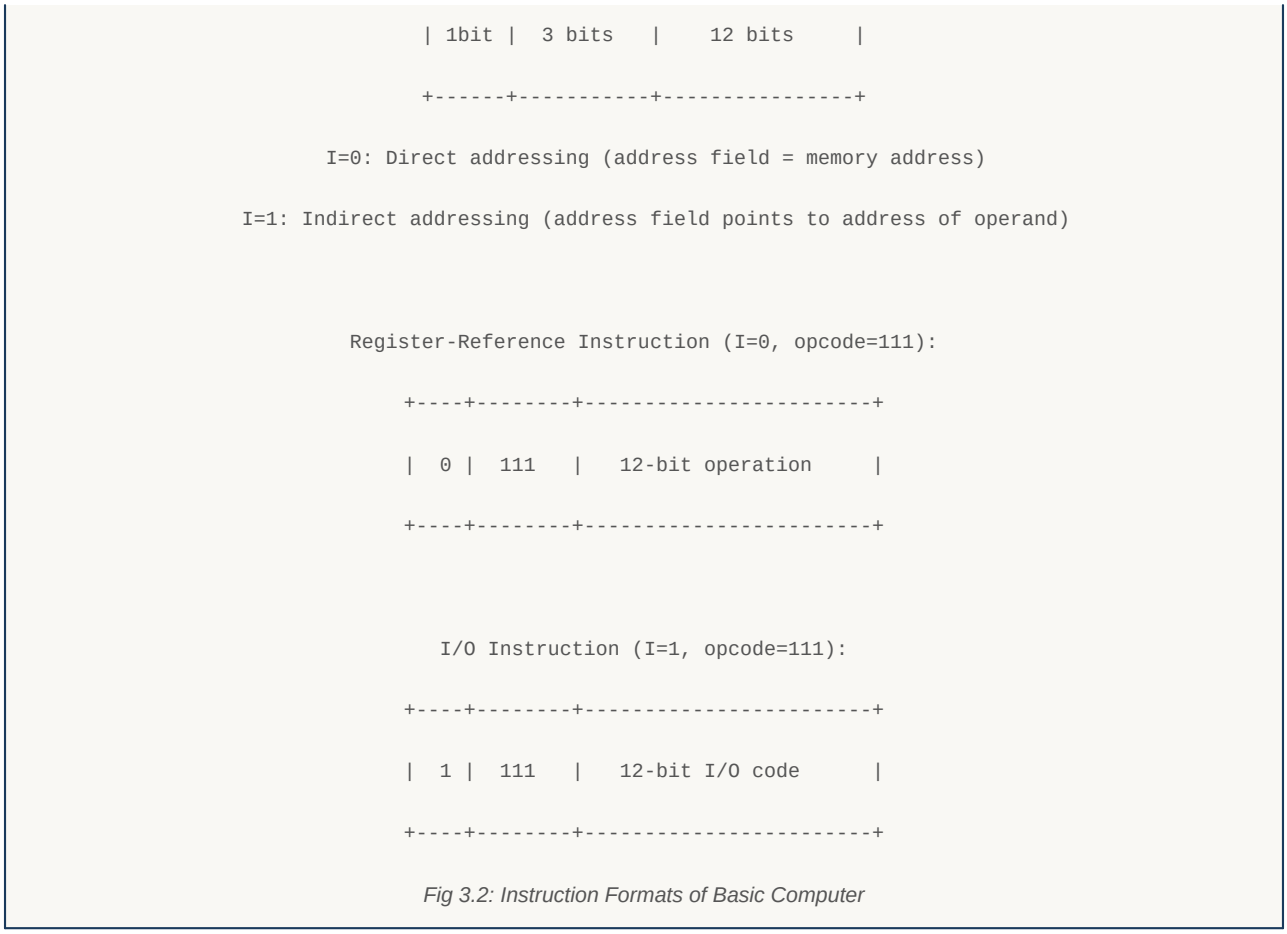


Fig 3.2: Instruction Formats of Basic Computer

Q2. Draw and Explain Instruction Cycle (Fetch, Decode, Execute)

The Instruction Cycle (also called Machine Cycle) is the complete process by which the CPU fetches, decodes, and executes an instruction. It repeats continuously as long as the computer is running.



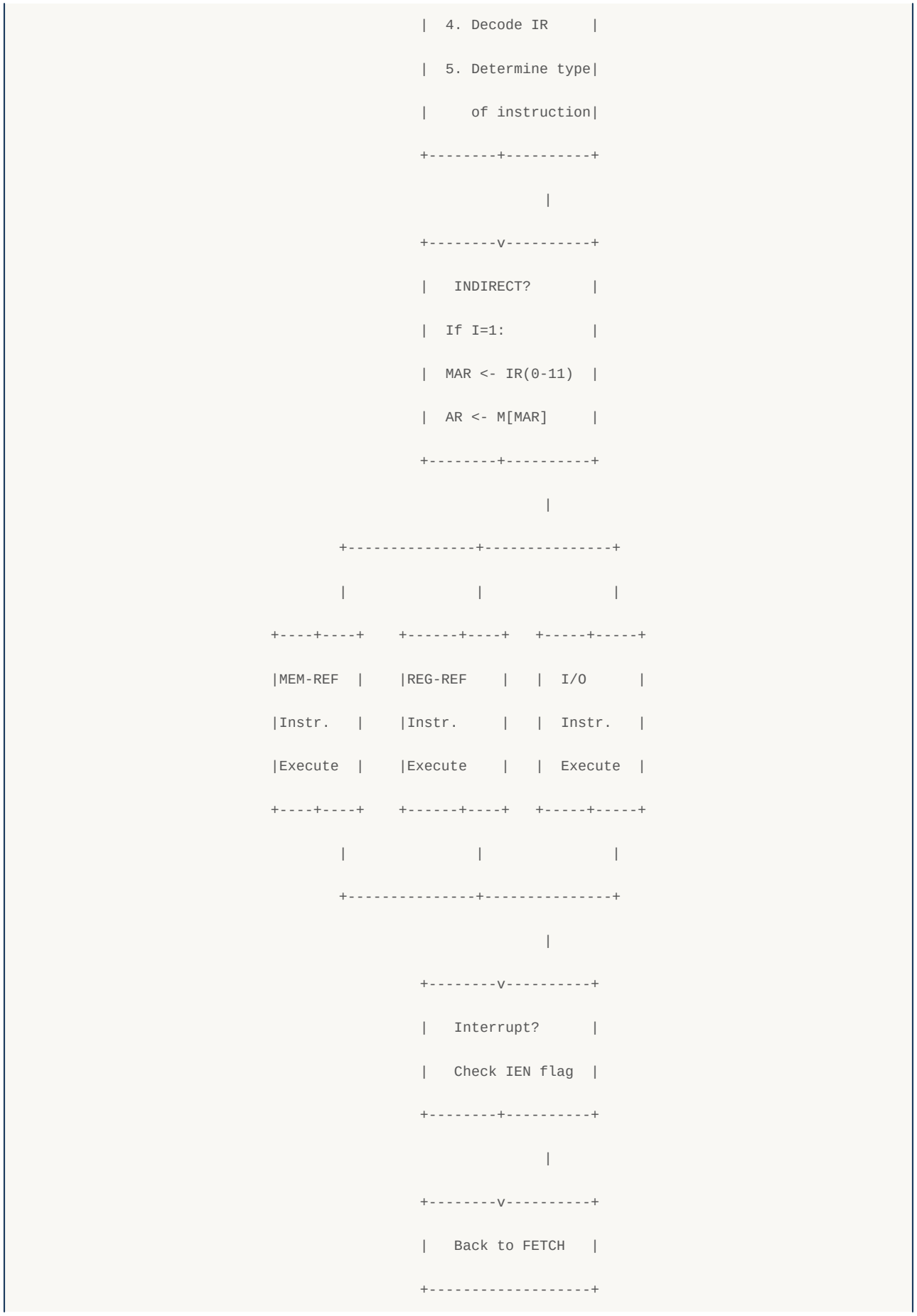

4. Decode IR
5. Determine type
of instruction
INDIRECT?
If I=1:
MAR <- IR(0-11)
AR <- M[MAR]
MEM-REF
Instr.
Execute
REG-REF
Instr.
Execute
I/O
Instr.
Execute
Interrupt?
Check IEN flag
Back to FETCH

Fig 3.3: Complete Instruction Cycle Flowchart

Detailed RTL for Each Phase:

1. Fetch Cycle (T0, T1, T2):

```
T0:  MAR <- PC              ; Transfer PC to MAR
T1:  MBR <- M[MAR], PC <- PC+1  ; Fetch instruction + increment PC
T2:  IR  <- MBR             ; Load instruction into IR
     Decode IR(12-14)        ; Decode opcode (bits 12-14)
     AR  <- IR(0-11)         ; Load address part
```

2. Decode and Execute — AND Instruction:

```
AND (AC <- AC AND M[AR]):
  If I=0 (Direct):
    T4: MAR <- AR
    T5: DR  <- M[MAR]
    T6: AC  <- AC AND DR
  If I=1 (Indirect):
    T3: MAR <- AR
        AR  <- M[MAR]        ; Get effective address
    T4-T6: Same as direct
```

3. Execute — ADD Instruction:

```
ADD (AC <- AC + M[AR]):
  T4: MAR <- AR
  T5: DR  <- M[MAR]
  T6: AC  <- AC + DR, E <- Carry_out
```

Q3. Memory-Reference Instructions with Examples

Memory-reference instructions reference a memory address. In the basic computer, there are 7 memory-reference instructions (opcode 000 to 110).

Opcode	Symbol	Operation	RTL Description
000	AND	Logical AND	AC ← AC ∧ M[AR]
001	ADD	Addition	AC ← AC + M[AR]; E=carry
010	LDA	Load to AC	AC ← M[AR]
011	STA	Store from AC	M[AR] ← AC
100	BUN	Branch Unconditional	PC ← AR
101	BSA	Branch & Save Return	M[AR]←PC; PC←AR+1
110	ISZ	Increment & Skip if Zero	M[AR]←M[AR]+1; if zero: PC←PC+1

Example — BSA (Branch and Save Return) used for Subroutine Calls:

```
BSA X:  M[X] <- PC     ; Save return address at location X
        PC  <- X + 1   ; Jump to subroutine (instruction after X)

To return from subroutine:
BUN X:  PC <- M[X]     ; Jump back to return address (indirect)
```

Q4. Input-Output and Interrupt Handling

I/O Instructions (Basic Computer):

I/O instructions are executed when opcode=111 and I=1. The 12-bit operation code determines the specific I/O operation.

Hex Code	Symbol	RTL	Description
E800	INP	AC(0-7) ← INPR; FGI ← 0	Input character into AC
E400	OUT	OUTR ← AC(0-7); FGO ← 0	Output character from AC
E200	SKI	If FGI=1: PC ← PC+1	Skip if input flag set
E100	SKO	If FGO=1: PC ← PC+1	Skip if output flag set
E040	ION	IEN ← 1	Enable interrupt
E020	IOF	IEN ← 0	Disable interrupt

Interrupt Handling:

An interrupt is a signal that temporarily halts the current program execution to handle a high-priority event (like I/O completion). The basic computer uses a single interrupt mechanism.

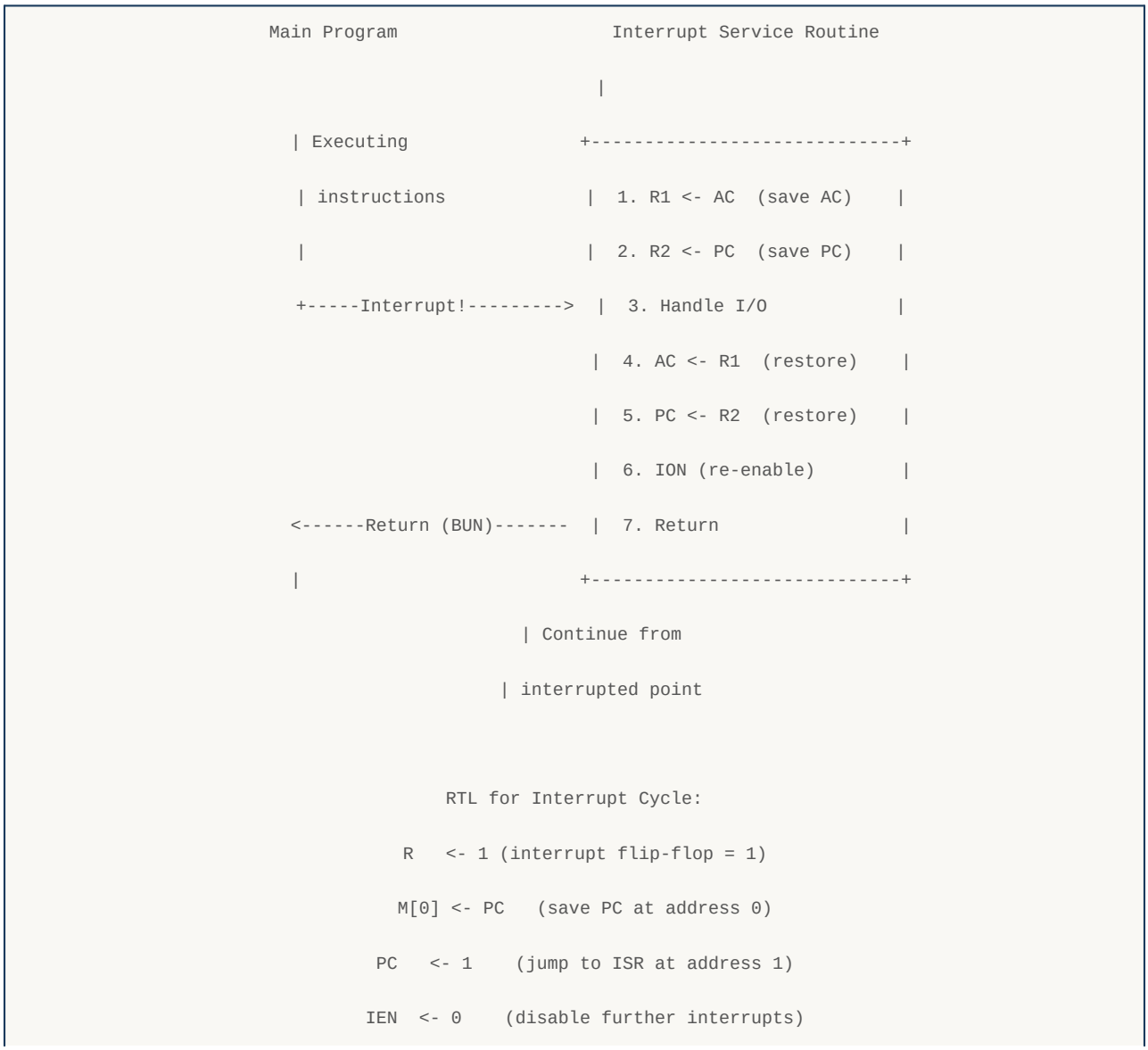


Fig 3.4: Interrupt Handling Mechanism

Q5. Design of Basic Computer / Accumulator Logic

The basic computer design uses an accumulator-based architecture. All arithmetic/logic operations are performed between the **Accumulator (AC)** and a memory operand.

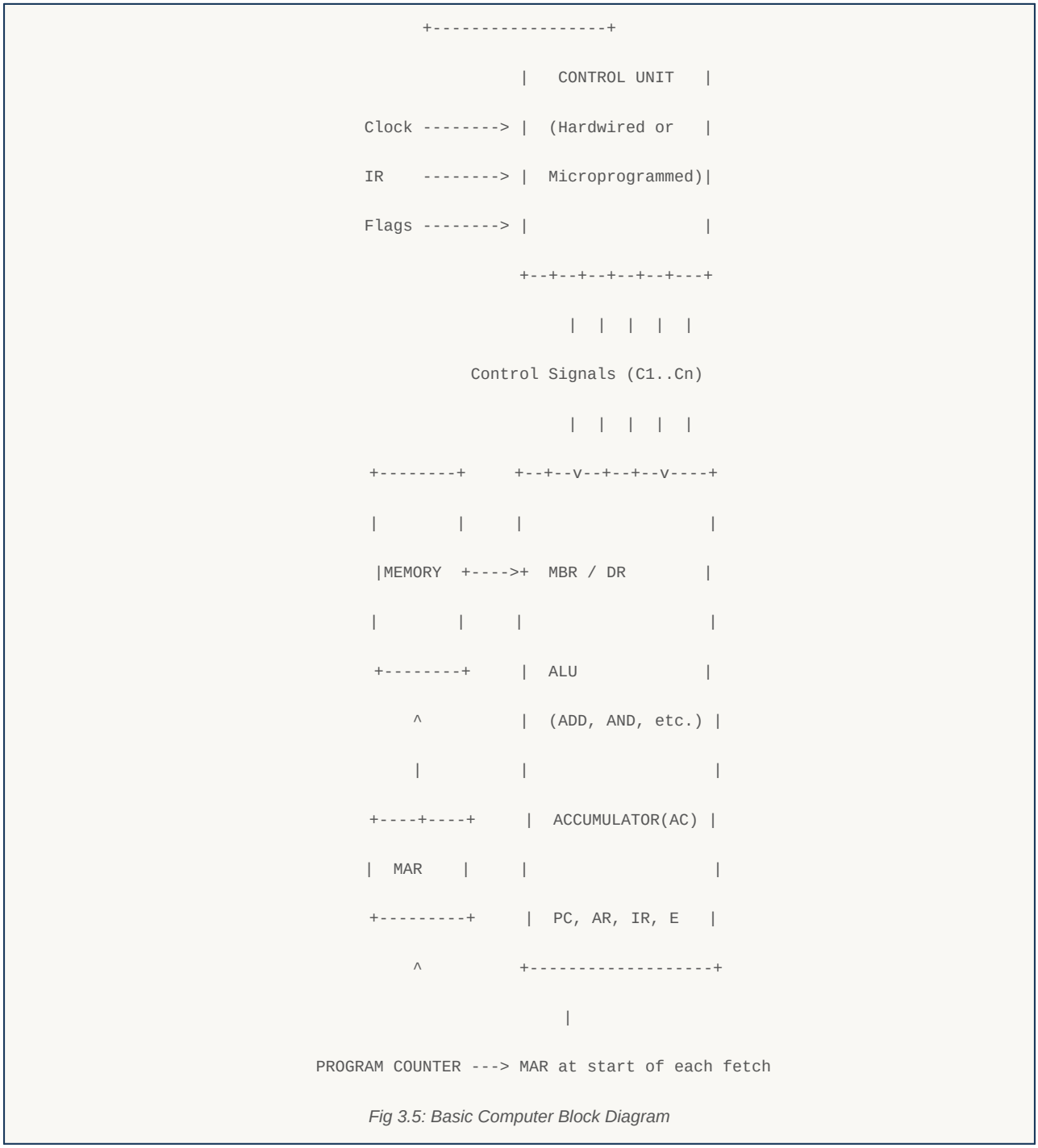


Fig 3.5: Basic Computer Block Diagram

Q6. Timing and Control Unit

The Control Unit generates all control signals that coordinate the operation of all units in the computer. The **Sequence Counter (SC)** generates timing pulses T0, T1, T2, ... Tn.

Hardwired Control Unit:

• Control signals generated by combinational logic circuits.
• Very fast but difficult to modify or expand.
• Fixed, dedicated hardware for each instruction.
• Used in RISC processors.

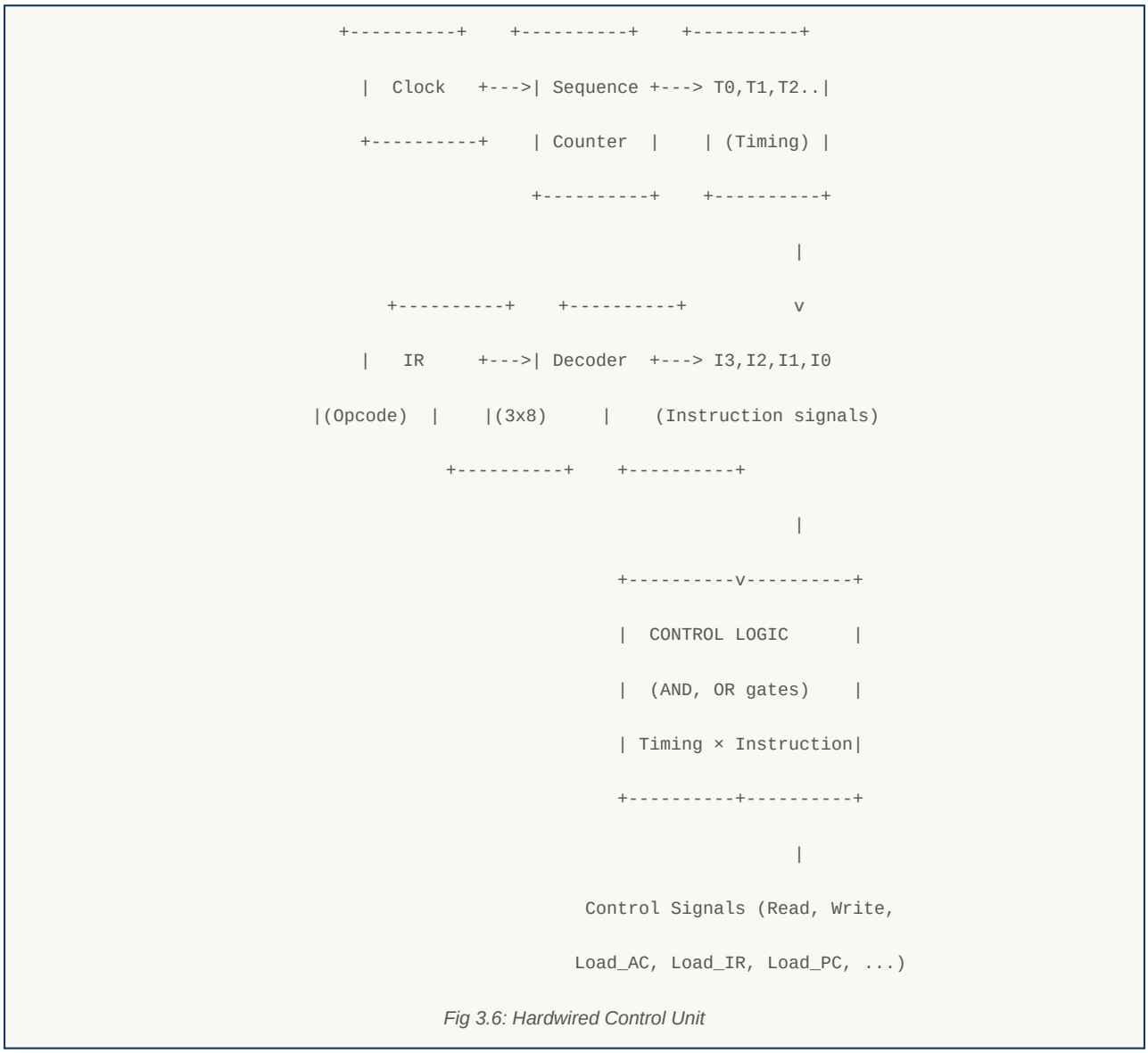


Fig 3.6: Hardwired Control Unit

Unit 4: Assembly Language Programming

6 Hours 10%

Q1. Machine Language vs Assembly Language

Feature	Machine Language	Assembly Language
Representation	Binary (0s and 1s)	Symbolic mnemonics (ADD, MOV, LDA)
Readability	Very difficult to read/write	Easier to read/write
Translation	Directly executed by CPU	Translated by Assembler to machine code
Speed	Fastest (no translation needed)	Slightly slower (assembly needed)
Portability	Not portable (CPU-specific)	Not portable (assembler-specific)
Debugging	Very difficult	Easier with symbolic names
Example	0001 0010 0000 0001	LDA X (Load address X)

Q2. Assembly Programs for Basic Computer

A) Addition of Two Numbers:

```
/ Add two numbers stored at memory locations A and B, store result at C
       ORG 100       / Start program at address 100 (hex)
       LDA A         / Load value at address A into AC
       ADD B         / Add value at address B to AC
       STA C         / Store result (in AC) into address C
       HLT           / Halt the program

/ Data Section
A,     DEC 10        / A = decimal 10
B,     DEC 20        / B = decimal 20
C,     DEC 0         / C = 0 (will hold result = 30)
       END

Trace:
  AC = 0 initially
  After LDA A: AC = 10
  After ADD B: AC = 10+20 = 30
  After STA C: Memory[C] = 30
```

B) Subtraction of Two Numbers:

```
/ Subtract B from A, store result at C  (C = A - B)
       ORG 100
       LDA A         / Load A into AC
       CMA           / Complement AC (bitwise NOT)
       INC           / Increment AC (now AC = -A in 2's complement)
       ADD B         / AC = B + (-A) = B - A
       / To get A - B:
       CMA           / Complement again
       INC           / AC = A - B
       STA C         / Store result
```

```
        HLT

A,      DEC 25
B,      DEC 10
C,      DEC 0          / Result will be 15
        END
```

C) Looping — Sum of N Numbers:

```
/ Calculate sum of first N numbers (1+2+3+...+N)
/ Assume N=5, numbers stored from address NUM onwards

        ORG 100
        LDA ZERO       / Initialize sum to 0
        STA SUM        / Store 0 in SUM
        LDA N          / Load counter N
        STA CTR        / Initialize counter
LOOP,   LDA SUM        / Load current sum
        ADD PTR        / Add current number (pointed by PTR)
        STA SUM        / Update sum
        ISZ PTR        / Increment pointer (PTR++)
        ISZ CTR        / Increment counter (CTR++)
        BUN LOOP       / If CTR not zero, repeat loop
        HLT            / Halt when done

N,      DEC 5          / Count of numbers
CTR,    DEC -5         / Counter (negative for ISZ trick)
PTR,    ORG NUM        / Pointer to number array
SUM,    DEC 0          / Sum storage
ZERO,   DEC 0
NUM,    DEC 1          / Numbers: 1,2,3,4,5
        DEC 2
        DEC 3
        DEC 4
        DEC 5
        END
```

D) Factorial using Subroutine:

```
/ Calculate N! using subroutine
        ORG 100
        LDA N          / Load N
        BSA FACT       / Call subroutine FACT (Branch & Save return)
        STA RESULT     / Store result
        HLT

/ Subroutine FACT: Computes N! (simple iterative)
FACT,   HEX 0          / Return address stored here
        STA CTR        / Store N as counter
        LDA ONE        / Load 1 (initial product)
FLOOP,  CMA            / Complement AC
        INC            / -AC
```

```
        ADD CTR       / subtract counter
        STA TEMP      / store temporarily
        LDA PROD      / load current product
        / (multiply logic simplified)
        ISZ CTR       / decrement counter
        BUN FLOOP     / loop until CTR=0
        BUN FACT I    / Return (indirect through FACT address)

N,      DEC 5
CTR,    DEC 0
PROD,   DEC 1
ONE,    DEC 1
RESULT,DEC 0
TEMP,   DEC 0
        END
```

E) I/O Programming — Input and Echo a Character:

```
/ Read a character from keyboard and print it on output
        ORG 100
START, SKI           / Skip if input flag (FGI) is set
        BUN START    / If not ready, keep checking (polling)
        INP          / Read character from keyboard into AC
        OUT          / Output AC to display
        HLT

/ More complete I/O with interrupt:
        ION          / Enable interrupt
MAIN,   ...          / Main program continues
        ...

/ Interrupt Service Routine at address 1:
        ORG 1
ISR,    BUN IHANDLE
IHANDLE, INP         / Read input
          OUT        / Echo output
          ION        / Re-enable interrupt
          BUN MAIN I  / Return to main (indirect)
```

Q3. Addressing Modes with Examples

Addressing modes specify how the operand of an instruction is accessed. Different modes provide flexibility in accessing data.

Mode	Effective Address	Description	Example
Immediate	Operand is in instruction itself	No memory access needed; fastest	ADD #5 (add 5 directly)
Direct	EA = Address field	Address field gives memory location directly	ADD 200 (add contents of loc 200)
Indirect	EA = M[Address field]	Address field points to location containing address	ADD @200 (200 contains address)
Register	EA = Register	Operand is in a CPU register	ADD R1 (add contents of R1)

Register Indirect	EA = M[Register]	Register contains address of operand	ADD (R1) (R1 has address)
Relative	EA = PC + Offset	Effective address relative to PC	ADD 100(PC)
Indexed	EA = Base + Index	Add index register to base address	ADD 200(X) [X=index]
Base Register	EA = Base Reg + Offset	Base register + constant offset	ADD 50(B) [B=base]

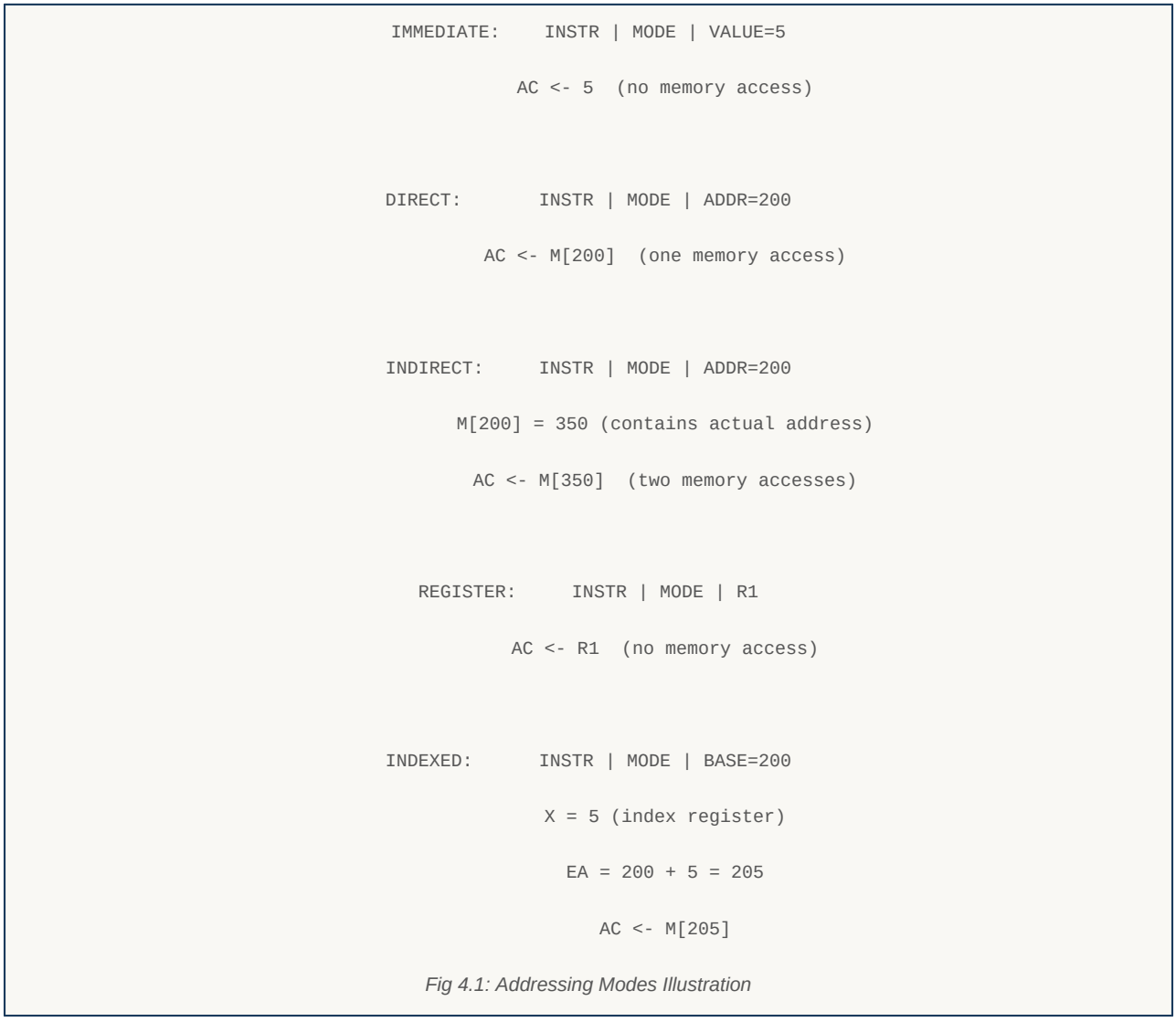


Fig 4.1: Addressing Modes Illustration

Unit 5: Micro-programmed Control Organization

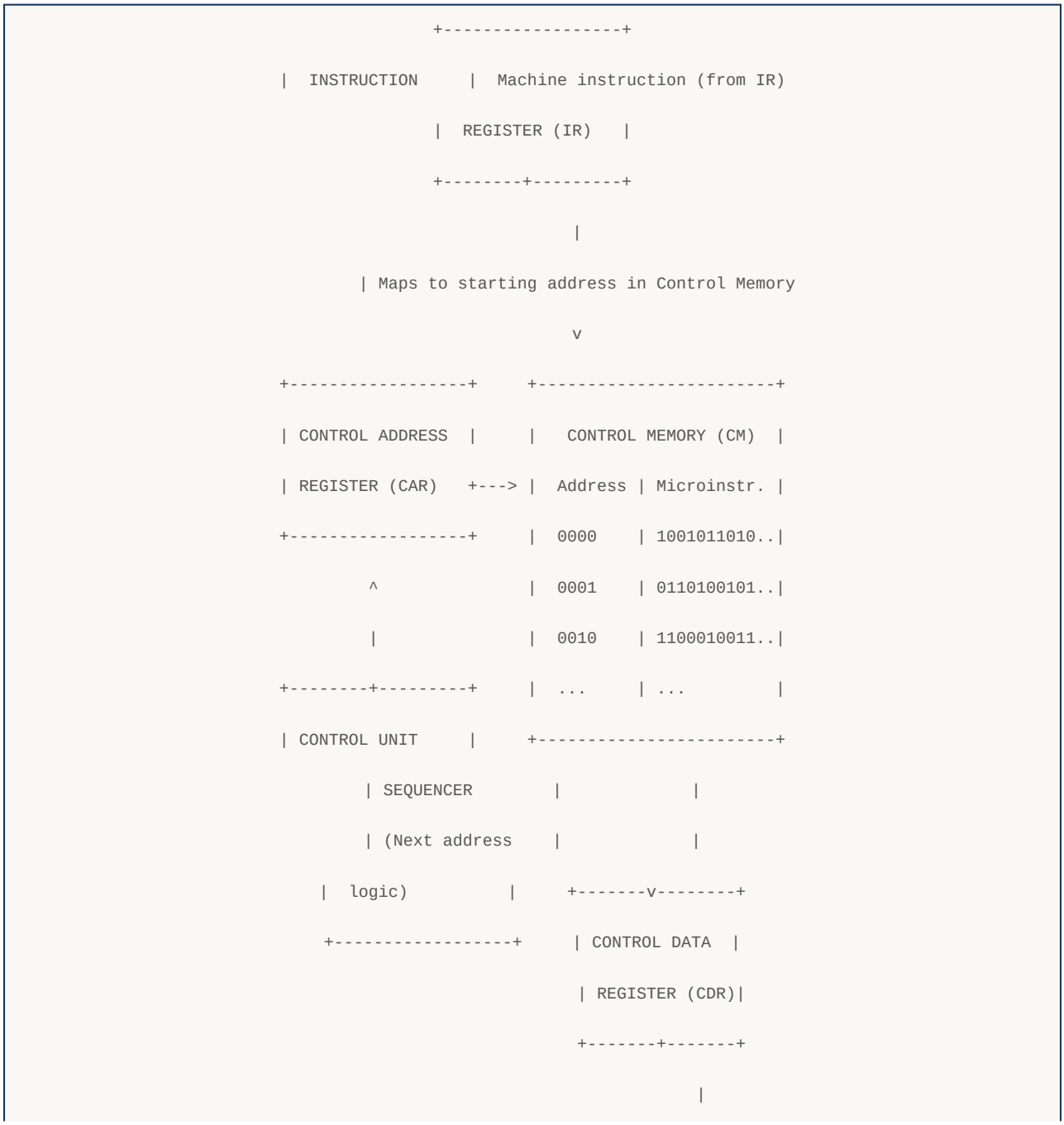
2 Hours 5%

Q1. Control Memory and Address Sequencing

In a micro-programmed control unit, each machine instruction is implemented by a sequence of micro-operations. These micro-operations are stored as **microinstructions** in a special high-speed memory called **Control Memory (CM)**.

Control Memory:

- Also called Control Store or Microprogram Memory.
- Stores microinstructions — each row is one microinstruction.
- Each microinstruction specifies micro-operations for one clock cycle.
- Typically implemented as ROM (read-only memory).
- Faster than main memory but slower than registers.



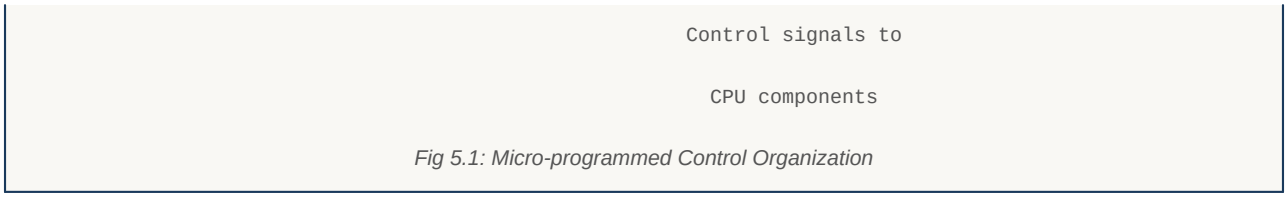


Fig 5.1: Micro-programmed Control Organization

Address Sequencing:

The next microinstruction address is determined by one of the following methods:

- **Sequential:** CAR = CAR + 1 (next address in sequence).
- **Unconditional Branch:** CAR = Address field of microinstruction.
- **Conditional Branch:** CAR = Address if condition true, else CAR+1.
- **Mapping:** Opcode of machine instruction maps to starting microinstruction address.
- **Subroutine Call:** CAR saved in subroutine register, branch to subroutine.

Q2. Micro-program Example

Microprogramming of Fetch Cycle:

```
Microinstruction Format:
+----------+----------+----------+------+------+--------+
| F1(3bit) | F2(3bit) | F3(3bit) | CD   | BR   | AD(7b) |
+----------+----------+----------+------+------+--------+
  F1,F2,F3: micro-operations (encoded)
  CD: condition for branch
  BR: branch type (JMP, CALL, RET, MAP)
  AD: address field

Microprogram for Fetch:
  Addr | F1  | F2  | F3  | CD | BR  | AD
  -----+-----+-----+-----+----+-----+----
  00   | PCTAR|    |     | U  | JMP | 01   ; AR <- PC
  01   | READ | INCPC|   | U  | JMP | 02   ; DR<-M[AR], PC<-PC+1
  02   | DRTIR|     |    | U  | MAP | 00   ; IR<-DR, map to opcode

Microprogram for ADD instruction:
  20   | ARTAR|     |    | U  | JMP | 21   ; AR <- AR (address from IR)
  21   | READ |     |    | U  | JMP | 22   ; DR <- M[AR]
  22   | ADD  |     |    | U  | RET | 00   ; AC <- AC+DR, return to fetch

  (PCTAR=PC to AR, READ=mem read, INCPC=PC++, DRTIR=DR to IR,
   ARTAR=address to AR, ADD=ALU add, MAP=map opcode, RET=return)
```

Q3. Hardwired vs Micro-programmed Control

Feature	Hardwired Control	Micro-programmed Control
Implementation	Random logic (gates, flip-flops)	Microprogram stored in control memory
Speed	Very fast	Slower (memory access per microinstruction)
Flexibility	Difficult to modify	Easy to modify (change microprogram)
Cost	Less for simple designs	More due to control memory

Complexity	Complex for complex instruction sets	Simpler design even for complex ISA
Design	Complex logic design	Write microprogram like software
Used in	RISC processors (simple ISA)	CISC processors (Intel x86)
Error correction	Need hardware redesign	Update microprogram in ROM

Q4. Advantages and Disadvantages of Micro-programmed Control

Advantages:

- Easy to design and implement even for complex instruction sets.
- Flexible — adding new instructions just requires adding microprograms.
- Errors can be corrected by modifying microprogram (not hardware).
- Systematic and organized design approach.
- Can implement CISC architectures with many complex instructions.
- Firmware updates possible without hardware changes.

Disadvantages:

- Slower than hardwired control due to memory accesses.
- Control memory adds cost to the system.
- Not suitable for high-speed RISC processors.
- Performance bottleneck when control memory is slow.
- More complex execution path per instruction.

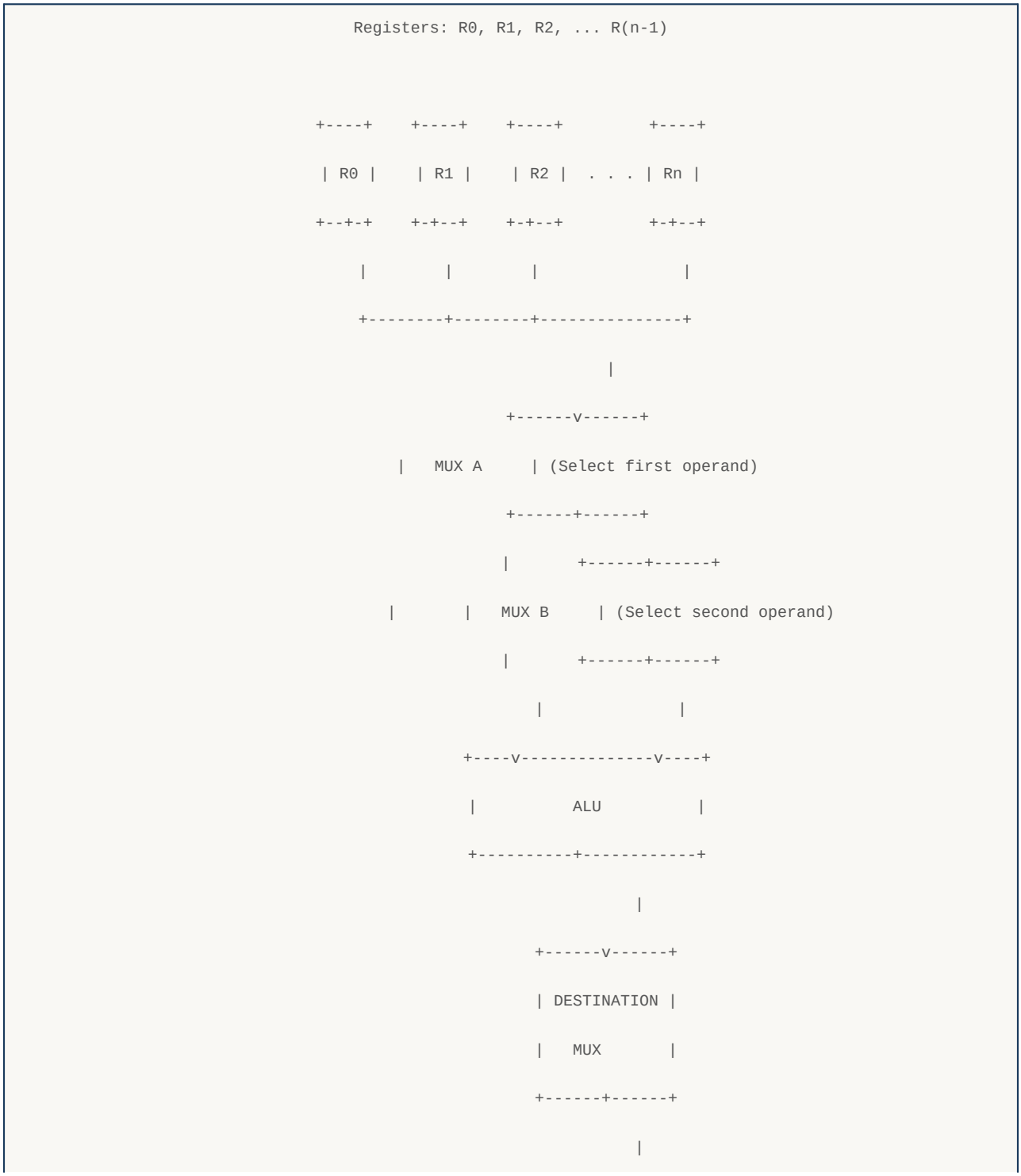
Unit 6: Central Processing Unit (CPU)

6 Hours 15% ★

> **IMPORTANT:** Very Important unit — 15% marks. RISC vs CISC is repeatedly asked in GTU. Master addressing modes and stack organization.

Q1. General Register Organization

In a General Register Organization, a set of registers (typically 8, 16, or 32) serve as fast temporary storage for operands and results. Any register can be used for any purpose — unlike accumulator-based designs.



```
                                Write result to any register
```

Fig 6.1: General Register Organization

- 3-address instruction: ADD R1, R2, R3 (R1 ← R2 + R3)
- 2-address instruction: ADD R1, R2 (R1 ← R1 + R2)
- 1-address instruction: ADD R2 (AC ← AC + R2)
- 0-address instruction: ADD (Pop two, push result — stack machine)

Q2. Stack Organization

A stack is a Last-In-First-Out (LIFO) data structure. Stack organization uses a hardware stack implemented with registers and a Stack Pointer (SP) that always points to the top of the stack.

```
                        PUSH operation: SP <- SP+1; M[SP] <- A

                        POP  operation: A <- M[SP]; SP <- SP-1



                                 Stack grows upward:

                                      +----------+

                          |    ..    |  <- SP (top after push)

                                      +----------+

                             |  data_3  |  <- SP points here

                                      +----------+

                                      |  data_2  |

                                      +----------+

                                |  data_1  |  <- bottom

                                      +----------+



                        Reverse Polish Notation (RPN) Example:

                                Expression: (3 + 4) * 5

                                     RPN: 3 4 + 5 *



                                PUSH 3        Stack: [3]

                               PUSH 4        Stack: [3, 4]

                           ADD (pop 4,3; push 7)  Stack: [7]

                               PUSH 5        Stack: [7, 5]

                           MUL (pop 5,7; push 35) Stack: [35]
```

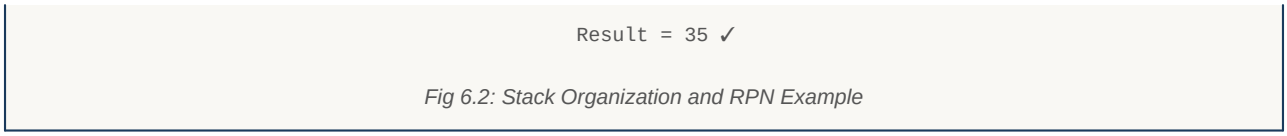


Fig 6.2: Stack Organization and RPN Example

Q3. Instruction Formats

Instruction format defines the layout of bits in an instruction — how opcode, operands, addressing mode, and other fields are organized.

Types of Instruction Formats:



```
Example: ADD  → TOS = TOS + NOS

(All operations on stack top)
```

Fig 6.3: Types of Instruction Formats

Q4. Addressing Modes (Detailed with Examples)

(Already covered in Unit 4 — extended here)

Example: Assume PC=200, AR=100, R1=400, M[100]=700, M[200]=3, M[400]=900, M[703]=800

Mode	Instruction	EA Calculation	Operand Value
Immediate	ADD #3	Operand = 3 (in instruction)	3
Direct	ADD 100	EA = 100	M[100] = 700
Indirect	ADD @100	EA = M[100] = 700	M[700] = ?
Register	ADD R1	EA = R1	R1 = 400
Register Indirect	ADD (R1)	EA = M[R1] = M[400]	M[400] = 900
Relative	ADD 3(PC)	EA = 200+3 = 203	M[203] = ?
Indexed	ADD 3(R1)	EA = R1+3 = 403	M[403] = ?
Auto-increment	ADD (R1)+	EA = M[R1]; R1++	M[400]=900; R1=401

Q5. RISC vs CISC

> **IMPORTANT:** This question is repeated in almost every GTU exam. Learn the table and characteristics thoroughly.

CISC — Complex Instruction Set Computer:

- Many complex instructions — each can do many operations.
- Instructions of variable length (1 to 15+ bytes).
- Many addressing modes (20+).
- Micro-programmed control unit.
- Fewer instructions needed to write a program.
- Example: Intel x86, Intel 8086, Motorola 68000.

RISC — Reduced Instruction Set Computer:

- Small set of simple instructions — each does one operation.
- Fixed instruction length (typically 32-bit).
- Few addressing modes (3-5).
- Hardwired control unit — very fast.
- Multiple register banks (32+ registers).
- Load-Store architecture: only LOAD and STORE access memory.
- Example: ARM, MIPS, SPARC, PowerPC, RISC-V.

Feature	RISC	CISC
Instruction set	Small, simple (~100)	Large, complex (300+)
Instruction length	Fixed (32-bit)	Variable (1-15 bytes)
Addressing modes	Few (3-5)	Many (20+)

Execution	1 clock cycle per instruction (ideally)	Multiple clock cycles
Control unit	Hardwired (fast)	Micro-programmed (slower)
Registers	Many (32+)	Few (8-16)
Pipelining	Very efficient	Difficult (variable length instr.)
Memory access	Load-Store only	Many instructions access memory
Code size	Larger (more instructions)	Smaller (complex instructions)
Power consumption	Lower	Higher
Examples	ARM, MIPS, SPARC, RISC-V	Intel x86, Intel 8086

Q6. Data Transfer and Manipulation Instructions

Data Transfer Instructions:

Instruction	Operation	Description
MOV R1, R2	R1 ← R2	Move (copy) register to register
LOAD R1, X	R1 ← M[X]	Load from memory to register
STORE R1, X	M[X] ← R1	Store register to memory
PUSH R1	SP--; M[SP]←R1	Push register onto stack
POP R1	R1←M[SP]; SP++	Pop from stack to register
XCHG R1,R2	R1↔R2	Exchange two registers
IN R1, port	R1 ← I/O[port]	Input from I/O port
OUT port, R1	I/O[port] ← R1	Output to I/O port

Program Control Instructions:

Instruction	Operation	Description
JMP ADDR	PC ← ADDR	Unconditional jump
JZ ADDR	If Z=1: PC←ADDR	Jump if zero flag set
JNZ ADDR	If Z=0: PC←ADDR	Jump if zero flag not set
JC ADDR	If C=1: PC←ADDR	Jump if carry flag set
JN ADDR	If N=1: PC←ADDR	Jump if negative
CALL SUB	Push PC; PC←SUB	Call subroutine
RET	Pop PC	Return from subroutine
NOP	No operation	Do nothing (1 cycle)
HLT	Stop CPU	Halt execution

Unit 7: Pipeline and Vector Processing

5 Hours ~10%

Q1. Flynn's Taxonomy

Flynn's Taxonomy (1966) classifies computer architectures based on the number of instruction streams and data streams processed simultaneously.

Class	Full Name	Description	Example
SISD	Single Instruction, Single Data	One instruction processes one data item at a time. Traditional sequential processor.	von Neumann, Single-core CPU
SIMD	Single Instruction, Multiple Data	One instruction applied to multiple data items simultaneously. Used in vector/array processors.	GPU, SSE/AVX extensions, Array processors
MISD	Multiple Instruction, Single Data	Multiple instructions on same data. Rarely used in practice.	Fault-tolerant systems (theoretical)
MIMD	Multiple Instruction, Multiple Data	Multiple processors, each running different instructions on different data.	Multiprocessors, Clusters, Multi-core CPUs

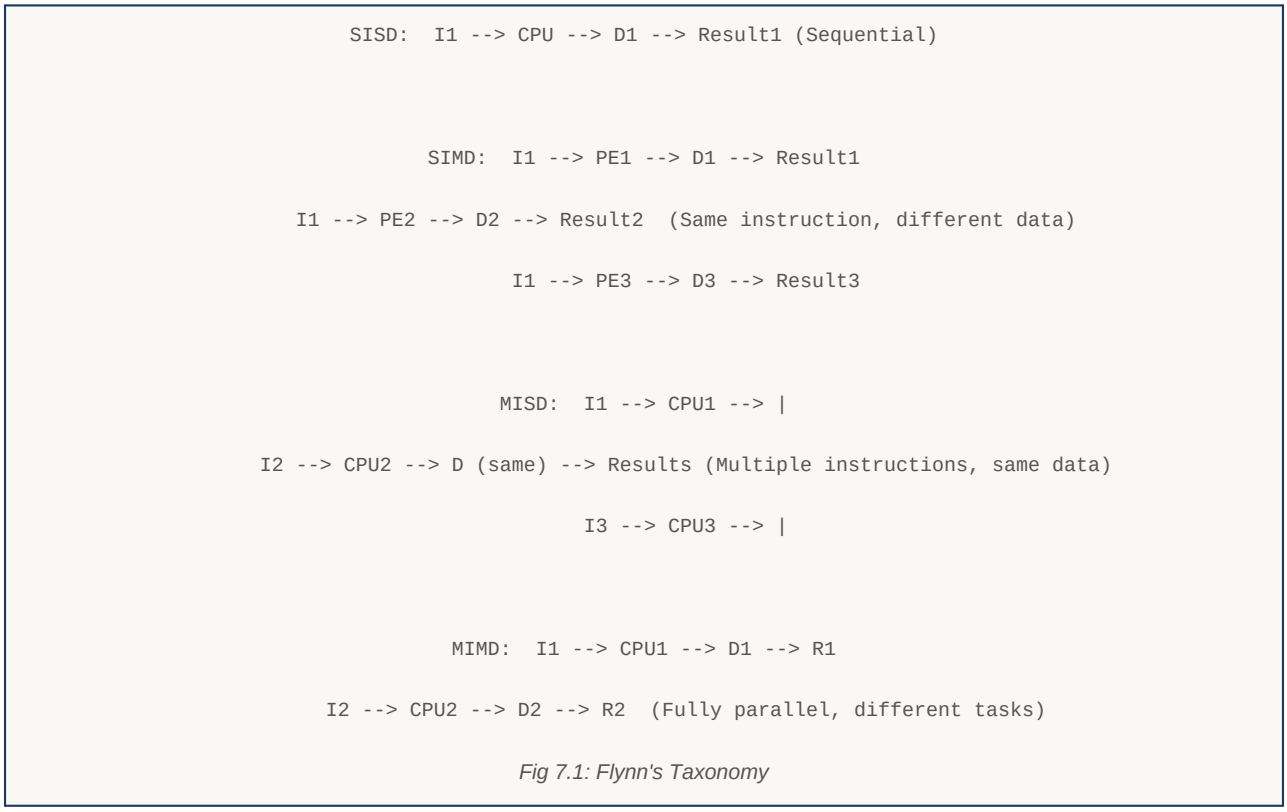
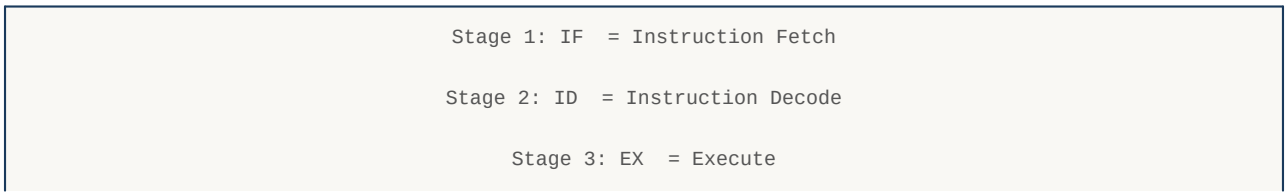


Fig 7.1: Flynn's Taxonomy

Q2. Pipelining (Instruction Pipeline, Arithmetic Pipeline)

Pipelining is a technique where multiple instructions are overlapped in execution. Like an assembly line — different stages of different instructions execute simultaneously.

Instruction Pipeline (4-Stage):



```
                    Stage 4: WB  = Write Back


          Time -->  T1   T2   T3   T4   T5   T6   T7

                 Instr 1: [IF] [ID] [EX] [WB]

                Instr 2:      [IF] [ID] [EX] [WB]

               Instr 3:           [IF] [ID] [EX] [WB]

              Instr 4:                [IF] [ID] [EX] [WB]


        Without pipeline: 4 instructions × 4 cycles = 16 cycles

        With pipeline:    4+3 = 7 cycles (speedup = 16/7 ≈ 2.3x)
```

Fig 7.2: Instruction Pipeline

Speedup Formula:

```
Speedup = (n × k) / (k + n - 1)
where n = number of instructions, k = number of stages

For large n: Speedup ≈ k (number of stages)

Example: 4-stage pipeline, 100 instructions
  Without: 100 × 4 = 400 cycles
  With:    4 + 100 - 1 = 103 cycles
  Speedup = 400/103 ≈ 3.88x
```

Arithmetic Pipeline — Floating Point Adder:

```
                FP Addition Pipeline (4 stages):

        Stage 1: Compare exponents (subtract exponents)

        Stage 2: Align mantissas (shift right smaller)

                Stage 3: Add mantissas

                Stage 4: Normalize result


              Input: A=(1.5 × 2^3), B=(1.25 × 2^3)


                Stage 1: Exp_A=3, Exp_B=3, Diff=0

              Stage 2: A=1.5, B=1.25 (no shift needed)

                    Stage 3: 1.5 + 1.25 = 2.75

                Stage 4: Normalize: 1.375 × 2^1
```

```
Output: 2.75 = 1.375 × 2^1 ✓
```

Fig 7.3: Arithmetic Pipeline for Floating Point Addition

Q3. Hazards in Pipelining and Solutions

Types of Hazards:

- **Data Hazard:** An instruction depends on result of a previous instruction still in pipeline.
- **Control Hazard (Branch Hazard):** Branch instruction changes PC — pipeline has already fetched wrong instructions.
- **Structural Hazard:** Two instructions need the same hardware resource at same time.

Data Hazard and Solutions:

```
RAW (Read After Write) — most common:
  ADD R1, R2, R3    ; R1 = R2 + R3 (writes R1 at WB stage)
  SUB R4, R1, R5    ; R4 = R1 - R5 (needs R1 at EX stage — HAZARD!)

Solutions:
1. Stalling (Bubbles): Insert NOP cycles to wait
   ADD R1, R2, R3
   NOP              ; Wait
   NOP              ; Wait
   SUB R4, R1, R5  ; Now safe

2. Data Forwarding (Bypassing):
   Forward result directly from EX/WB stage to next EX stage
   No NOPs needed — hardware detects and forwards

3. Out-of-Order Execution:
   Reorder instructions to avoid hazard (if no dependency)
```

Control Hazard — Branch Prediction:

```
Problem: When BRANCH instruction is decoded, next 2-3 instructions
         are already in pipeline but may be wrong.

Solutions:
1. Flush pipeline (discard fetched instructions) — slow
2. Delayed Branching: Always execute next instruction(s) after branch
3. Branch Prediction:
   - Static: Always predict NOT taken (continue sequential)
   - Dynamic: Use history table to predict based on past behavior
   - Modern CPUs achieve ~95% accuracy in branch prediction
```

Q4. RISC Pipeline

RISC processors are designed specifically for efficient pipelining. The simple, fixed-length instructions make pipeline implementation straightforward.

```
RISC 5-Stage Pipeline (MIPS):
```

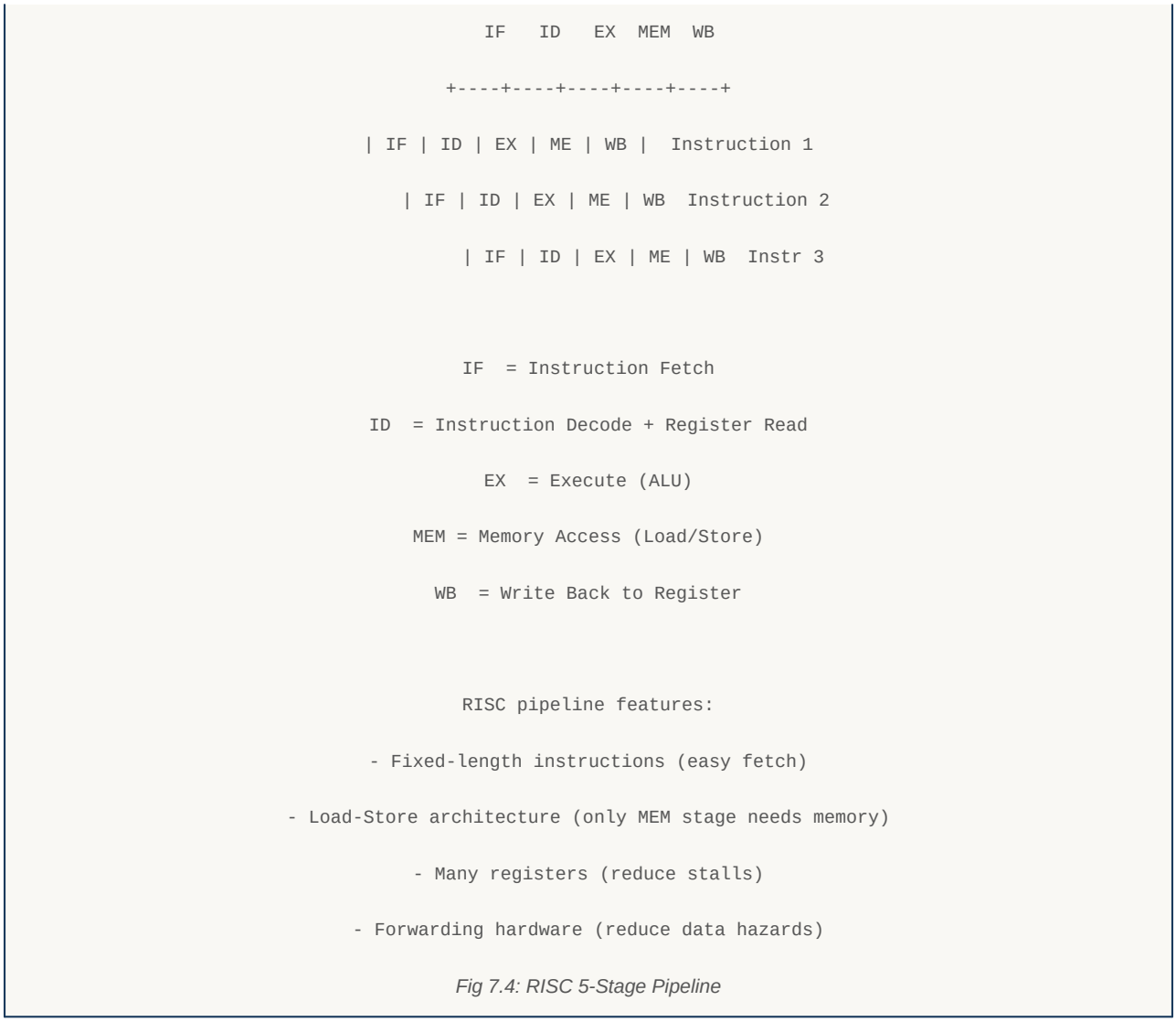


Fig 7.4: RISC 5-Stage Pipeline

Q5. Vector Processing and Array Processors

Vector Processing:

Vector processors operate on entire arrays (vectors) of data with a single instruction, instead of processing one element at a time.

```
Scalar (traditional):
  for(i=0; i<100; i++)
    C[i] = A[i] + B[i];   // 100 separate ADD operations

Vector (single instruction):
  C = A + B;              // One VADD instruction — process all 100 elements
                          // in parallel using vector hardware units
```

Array Processors:

An array processor is a parallel processing system consisting of multiple identical processing elements (PEs) that simultaneously execute the same instruction on different data (SIMD). All PEs receive same instruction from a single control unit.



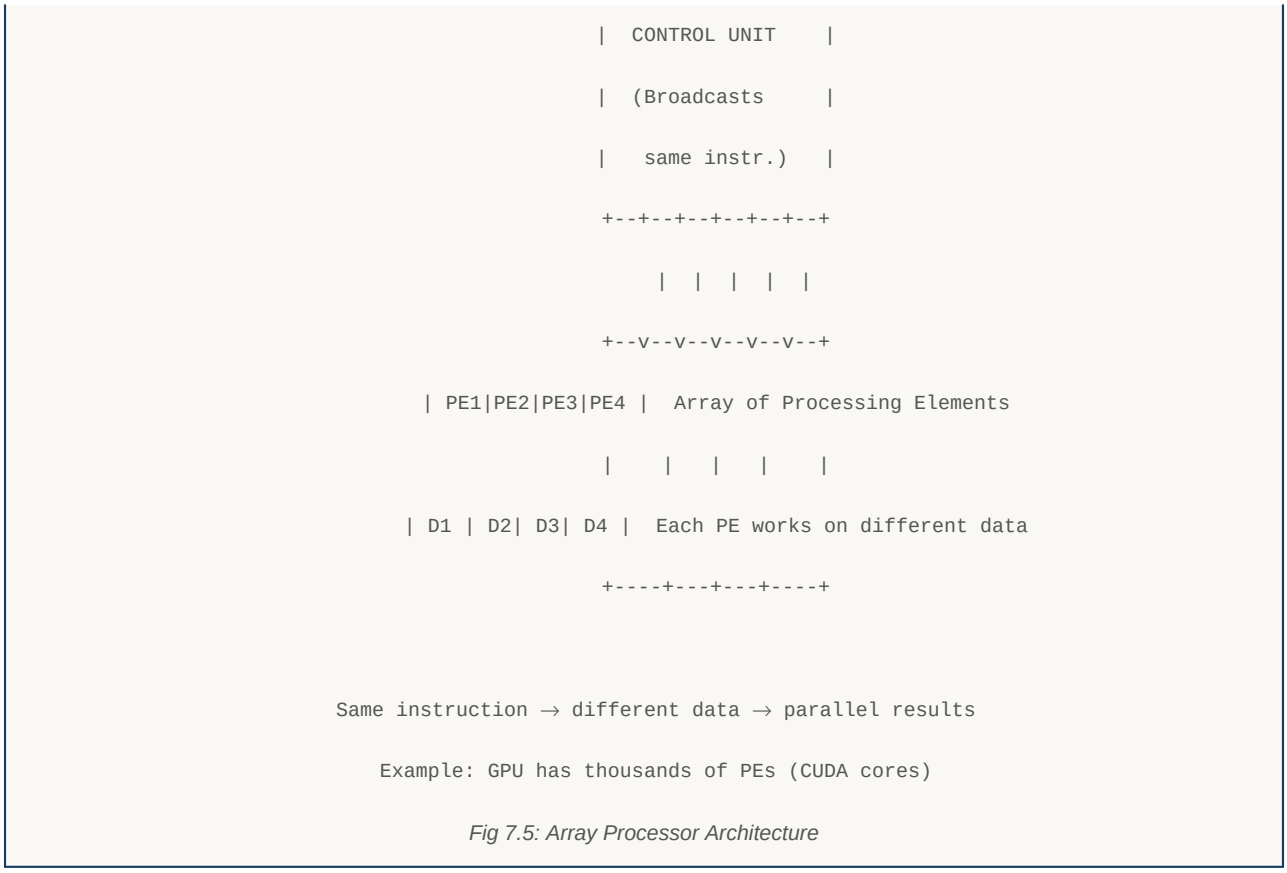


Fig 7.5: Array Processor Architecture

Unit 8: Input-Output Organization

4 Hours 10%

Q1. I/O Interface

An I/O interface provides the communication pathway between the CPU/memory and I/O devices. It handles speed differences, data format differences, and synchronization between the CPU and devices.

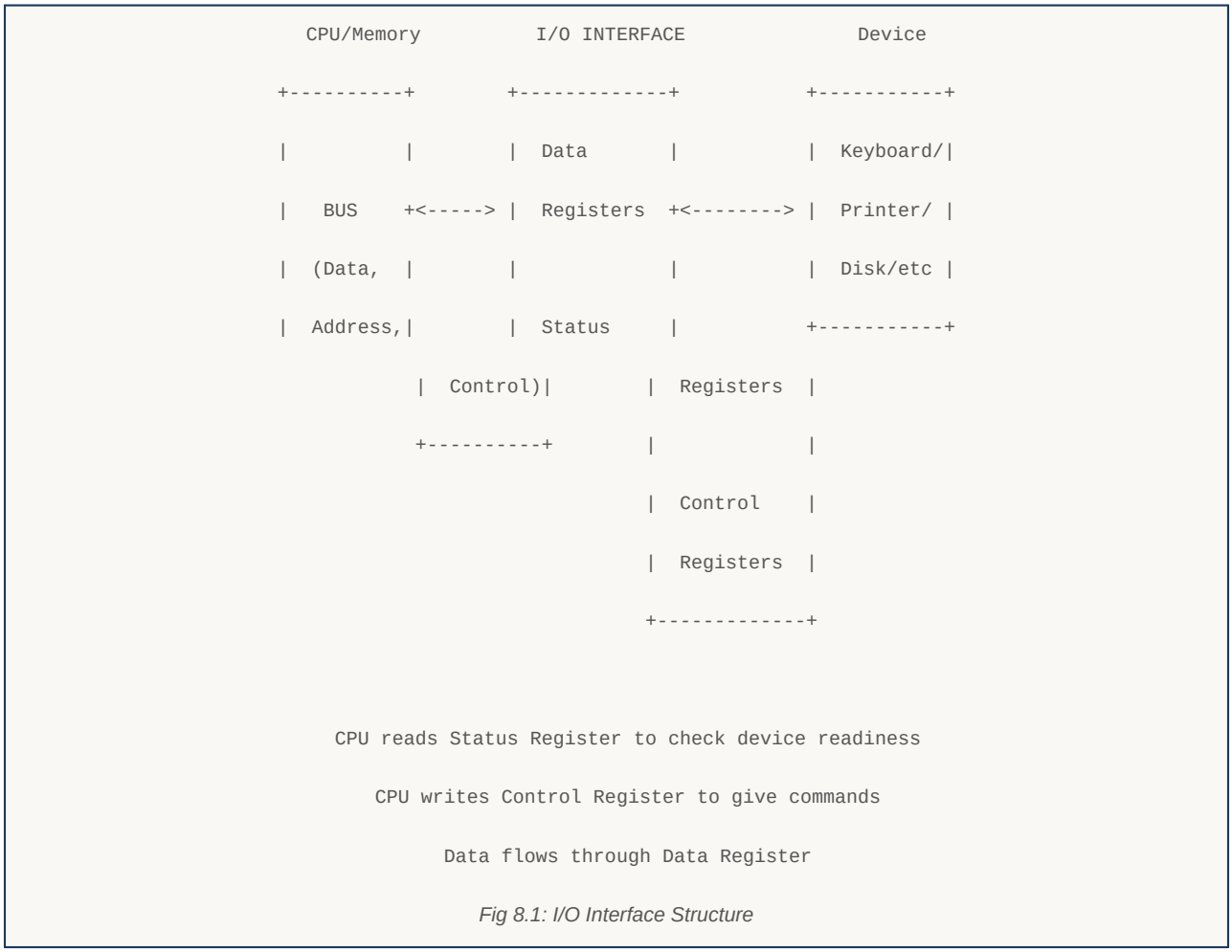
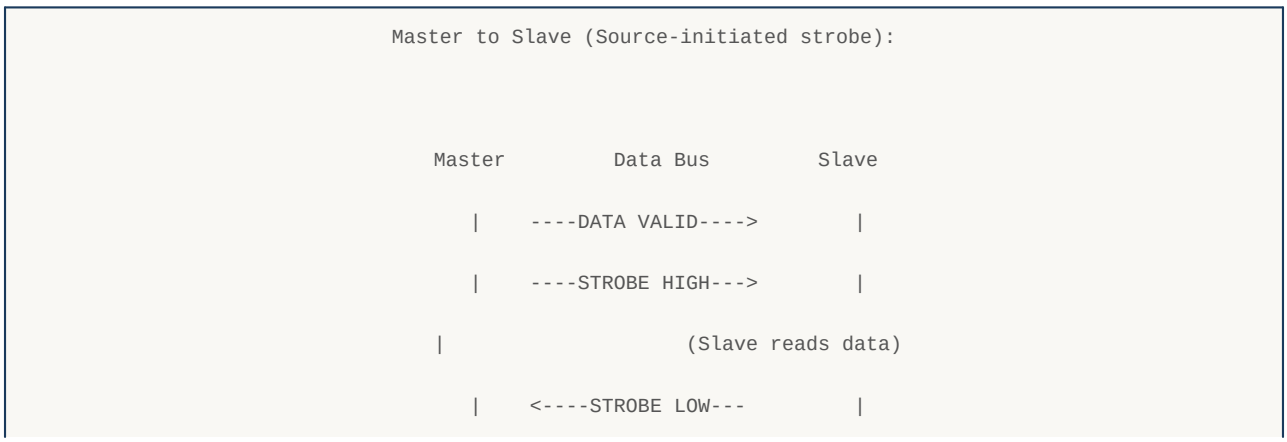


Fig 8.1: I/O Interface Structure

Q2. Asynchronous Data Transfer

Strobe Control:

A strobe signal is a single control line used to signal when data is valid on the bus. One side controls the strobe.



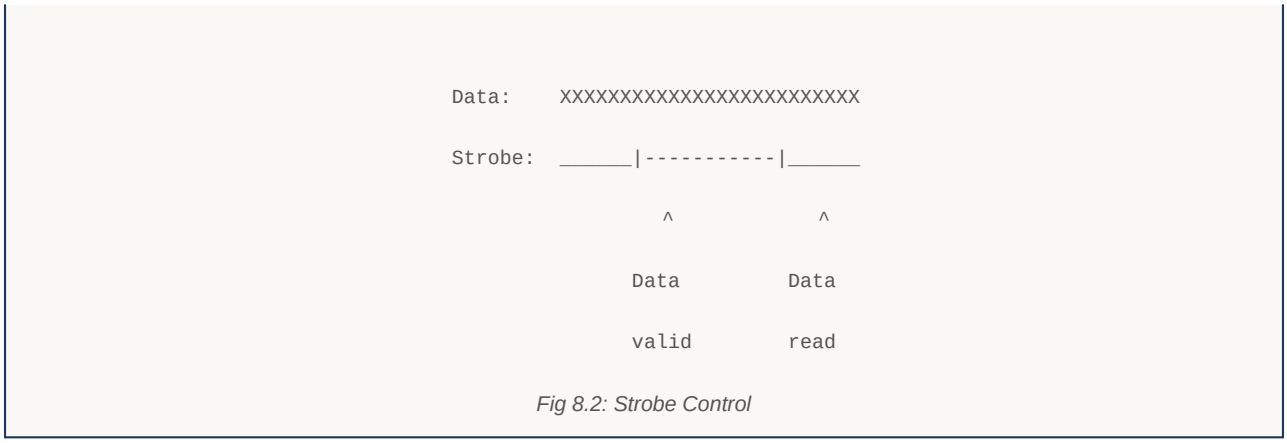


Fig 8.2: Strobe Control

Handshaking:

Two control signals are used — both source and destination participate in the handshake. More reliable than strobe.

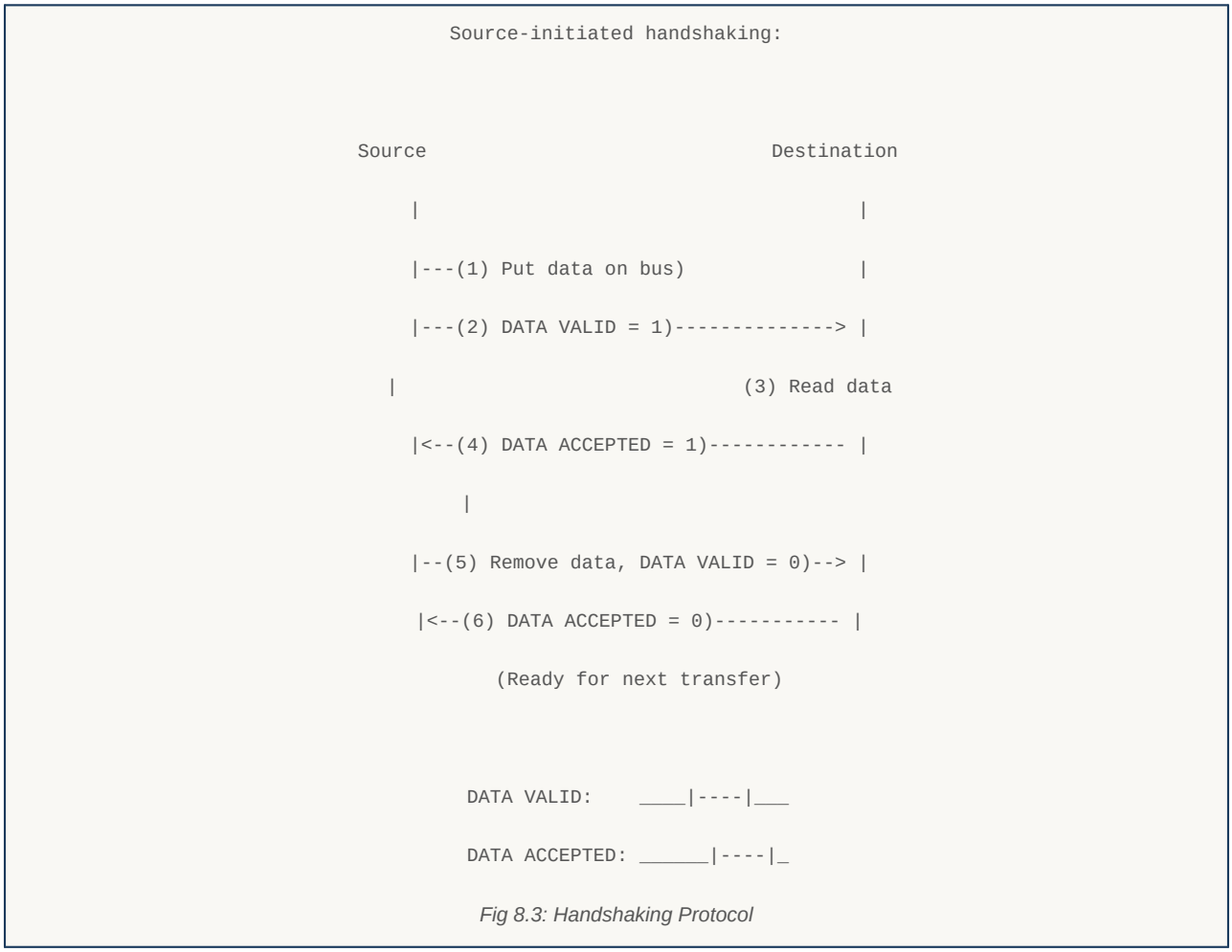


Fig 8.3: Handshaking Protocol

Q3. Modes of Transfer

A) Programmed I/O (Polling):

CPU directly controls I/O. CPU continuously checks (polls) status register of device until it is ready. Also called Busy-wait I/O.

```
  Algorithm:
    repeat:
      READ Status_Register
```

```
    if Status == READY:
      transfer data
      break
    else:
      loop (keep checking)

Advantage: Simple implementation
Disadvantage: CPU wastes time polling — cannot do other work
```

B) Interrupt-Driven I/O:

CPU starts I/O operation and then continues with other tasks. Device sends interrupt when it is ready, CPU then handles data transfer.

```
                    CPU starts I/O → continues with other tasks
                                          ↓
                              Device completes → INTERRUPT
                                          ↓
                          CPU saves context → jumps to ISR
                                          ↓
                                ISR transfers data
                                          ↓
                          CPU restores context → continues


                        Advantage: CPU is not idle — better utilization
                    Disadvantage: Interrupt overhead; still CPU involved in transfer
```

C) Direct Memory Access (DMA):

DMA allows I/O devices to transfer data directly to/from memory without CPU involvement. CPU only initiates and terminates the transfer.

```
          WITHOUT DMA:  Device → CPU → Memory  (CPU involved in every byte)


            WITH DMA:     Device → DMA Controller → Memory  (CPU free)


                                  DMA Operation:
                        1. CPU programs DMA (address, count, direction)
                        2. CPU starts DMA transfer and goes back to work
                     3. DMA controller takes bus control (HOLD/HLDA signals)
```

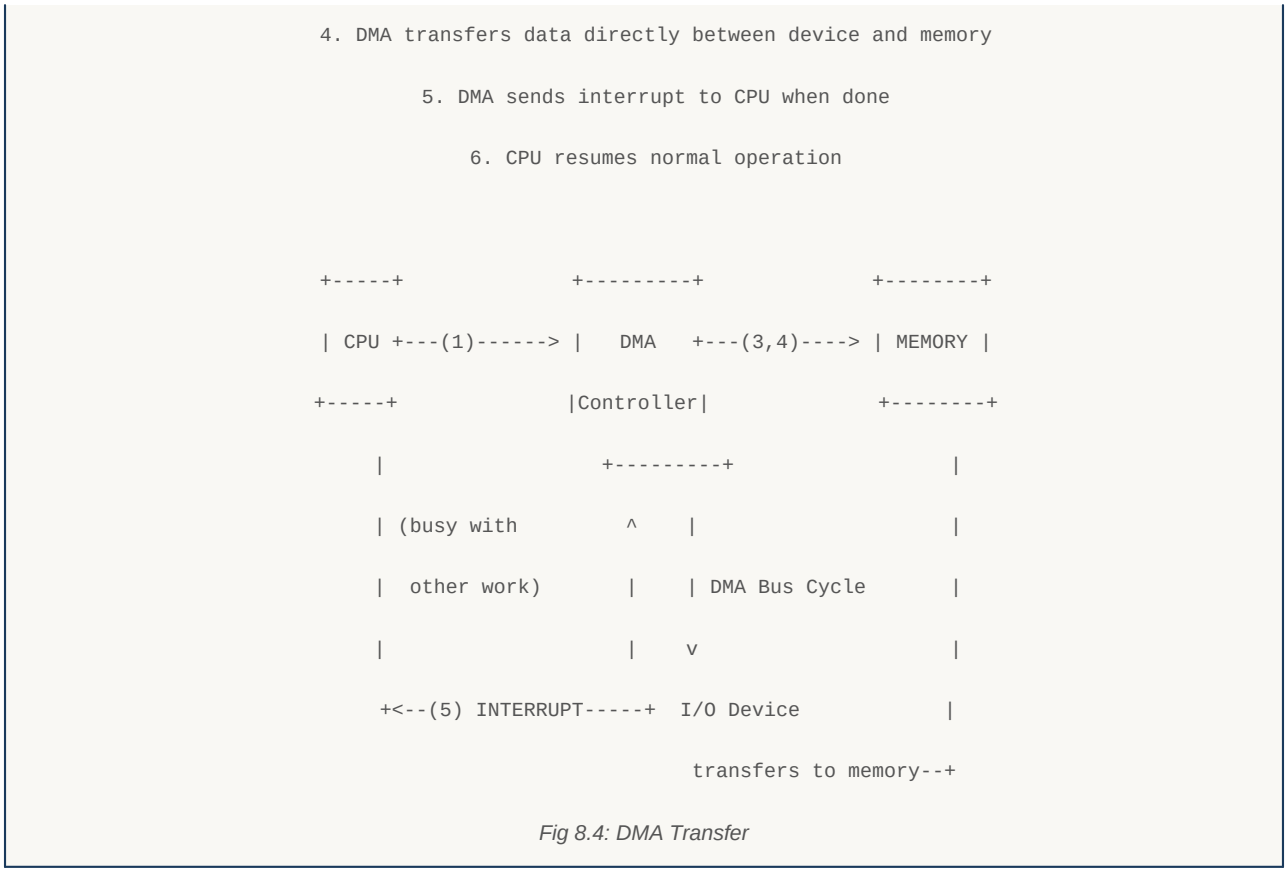


Fig 8.4: DMA Transfer

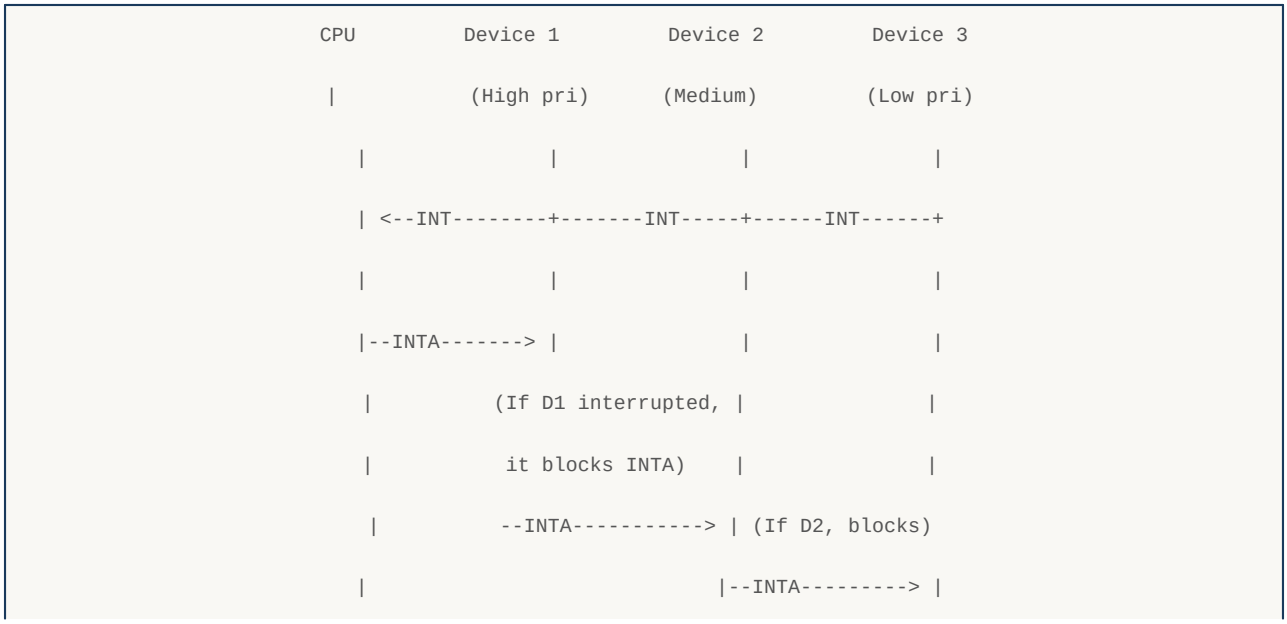
Q4. Priority Interrupt and Daisy Chaining

Priority Interrupt:

When multiple devices can interrupt simultaneously, the system must decide which to service first. Priority interrupt assigns priority levels to each device.

Daisy Chaining:

Hardware method for priority interrupts. Interrupt acknowledge signal is passed through devices in a chain. Highest priority device is first in chain.



```
First device in chain has highest priority
It consumes INTA signal if it raised INT
Otherwise passes INTA to next device
```

Fig 8.5: Daisy Chain Priority Interrupt

Q5. I/O Processor (IOP) and Serial Communication

I/O Processor (IOP):

An IOP (also called I/O Channel or DMA Controller with intelligence) is a specialized processor that handles I/O operations independently. It has its own instruction set for I/O operations.

- IOP fetches I/O instructions from memory (called channel program).
- Executes I/O operations independently of main CPU.
- Reports completion to CPU via interrupt.
- Examples: IBM System/360 I/O channels, Intel 8089.

Serial Communication:

- **Serial transmission:** Data bits sent one at a time over a single wire.
- **Parallel transmission:** Multiple bits sent simultaneously over multiple wires.
- **UART (Universal Asynchronous Receiver/Transmitter):** Standard for asynchronous serial communication. Uses start bit, data bits, optional parity, stop bit(s).
- **Baud Rate:** Number of signal changes per second. For binary: baud rate = bit rate.

```
Serial Frame (UART):
  [START=0][D0][D1][D2][D3][D4][D5][D6][D7][PARITY][STOP=1]

  Start bit (0): signals beginning of frame
  8 data bits: actual data
  Parity bit: error detection
  Stop bit (1): signals end of frame

  Common settings: 9600 baud, 8 data bits, No parity, 1 stop bit (9600-8N1)
```

Unit 9: Memory Organization

4 Hours 10% ★

Q1. Memory Hierarchy

Memory hierarchy organizes storage systems into levels based on speed, cost, and size. Faster memory is smaller and more expensive; slower memory is larger and cheaper.

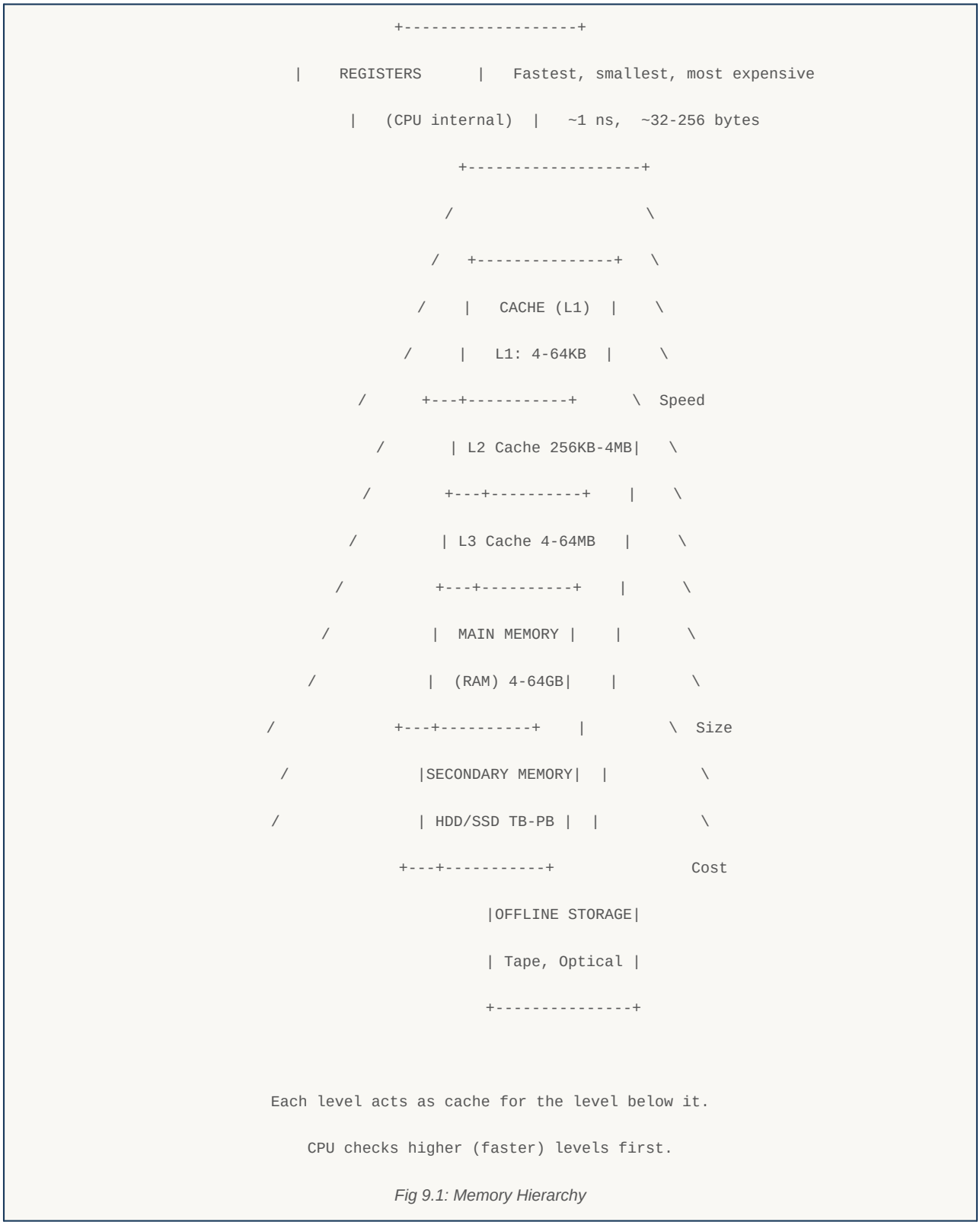


Fig 9.1: Memory Hierarchy

Level	Type	Access Time	Size	Cost
L1	Registers	< 1 ns	< 1 KB	Highest
L2	Cache (L1)	1-5 ns	4-64 KB	Very High
L3	Cache (L2)	5-20 ns	256KB-4MB	High
L4	Cache (L3)	20-50 ns	4-64 MB	Medium-High
L5	Main RAM	50-100 ns	4-64 GB	Medium
L6	SSD	50-100 µs	256GB-4TB	Low
L7	HDD	5-10 ms	1-20 TB	Very Low

Q2. Main Memory (RAM, ROM) vs Auxiliary Memory

Feature	RAM	ROM	Auxiliary Memory
Full Name	Random Access Memory	Read-Only Memory	Secondary/Backup Storage
Volatility	Volatile (loses data on power off)	Non-volatile (retains data)	Non-volatile
Access	Read and Write	Read only	Sequential or Direct
Speed	Very fast (~100 ns)	Fast (~100 ns)	Slow (ms for HDD)
Use	Running programs, OS	BIOS, boot code, firmware	Long-term data storage
Types	SRAM (cache), DRAM (main)	PROM, EPROM, EEPROM, Flash	HDD, SSD, Tape, Optical
Cost	High	Medium	Low

Q3. Cache Memory — Mapping Techniques

Cache is a small, fast memory between CPU and main memory. When CPU needs data, it first checks cache. If found (cache hit) — fast; if not (cache miss) — fetch from main memory and store in cache.

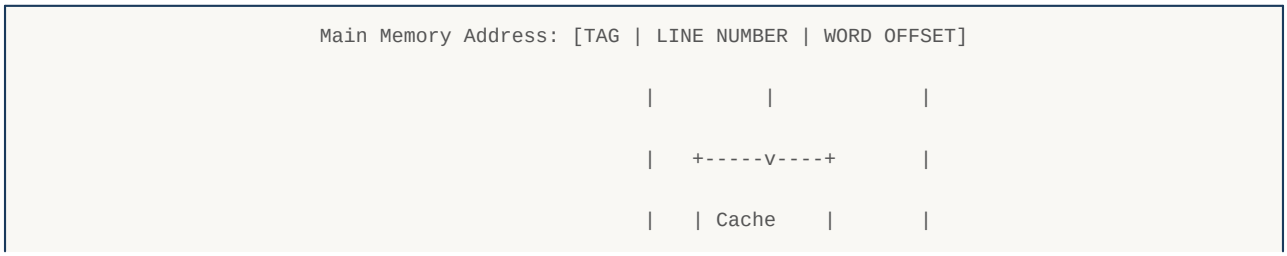
Performance:

```
Cache Hit Ratio (h) = Cache Hits / Total Memory Accesses
Average Access Time = h × Tc + (1-h) × Tm
  where Tc = cache access time, Tm = main memory access time

Example: h=0.9, Tc=10ns, Tm=100ns
  Average = 0.9×10 + 0.1×100 = 9 + 10 = 19 ns
  (vs 100ns without cache – 5x improvement!)
```

A) Direct Mapping:

Each block of main memory maps to exactly one cache line. Simple but may cause conflicts.



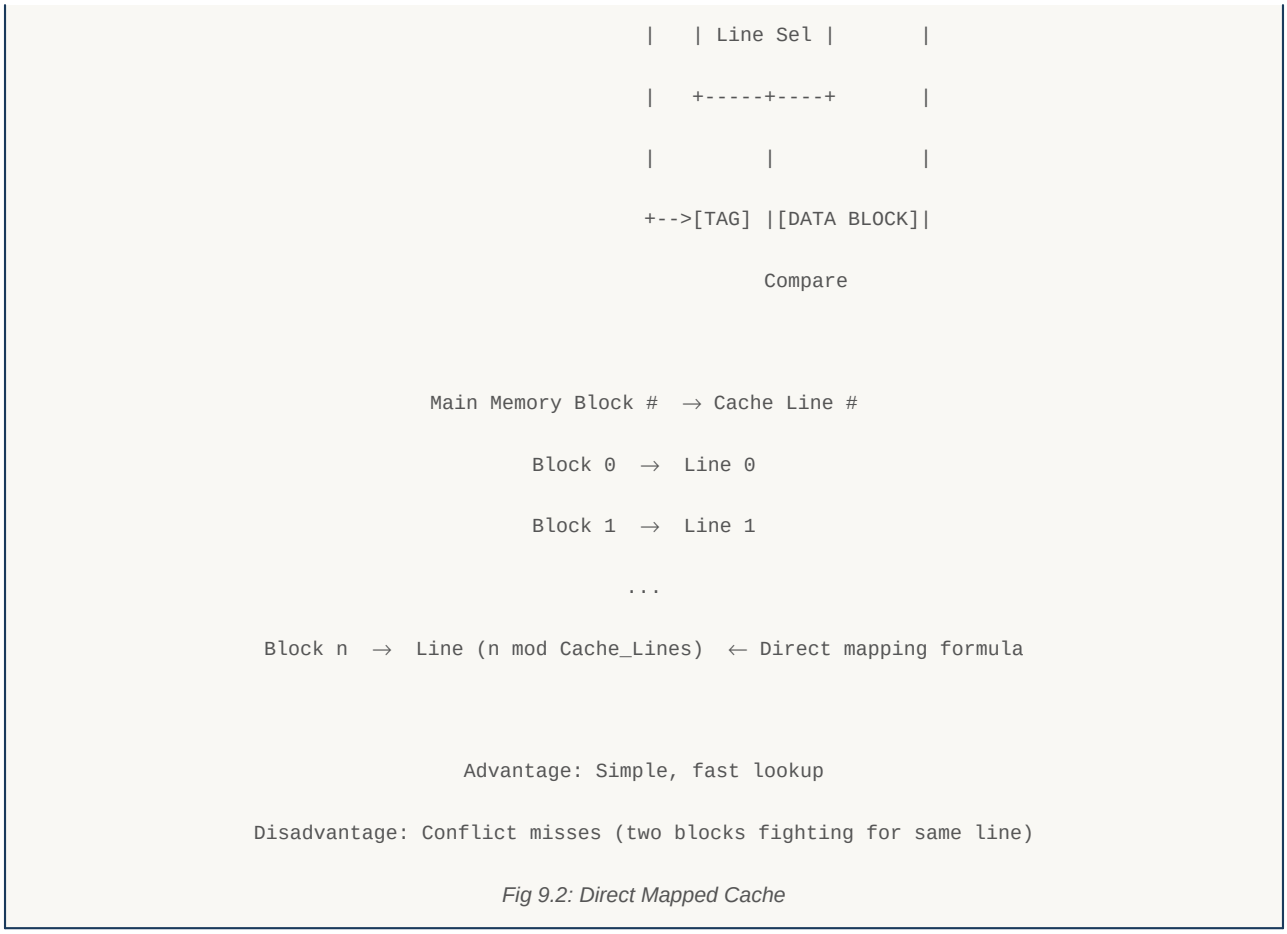
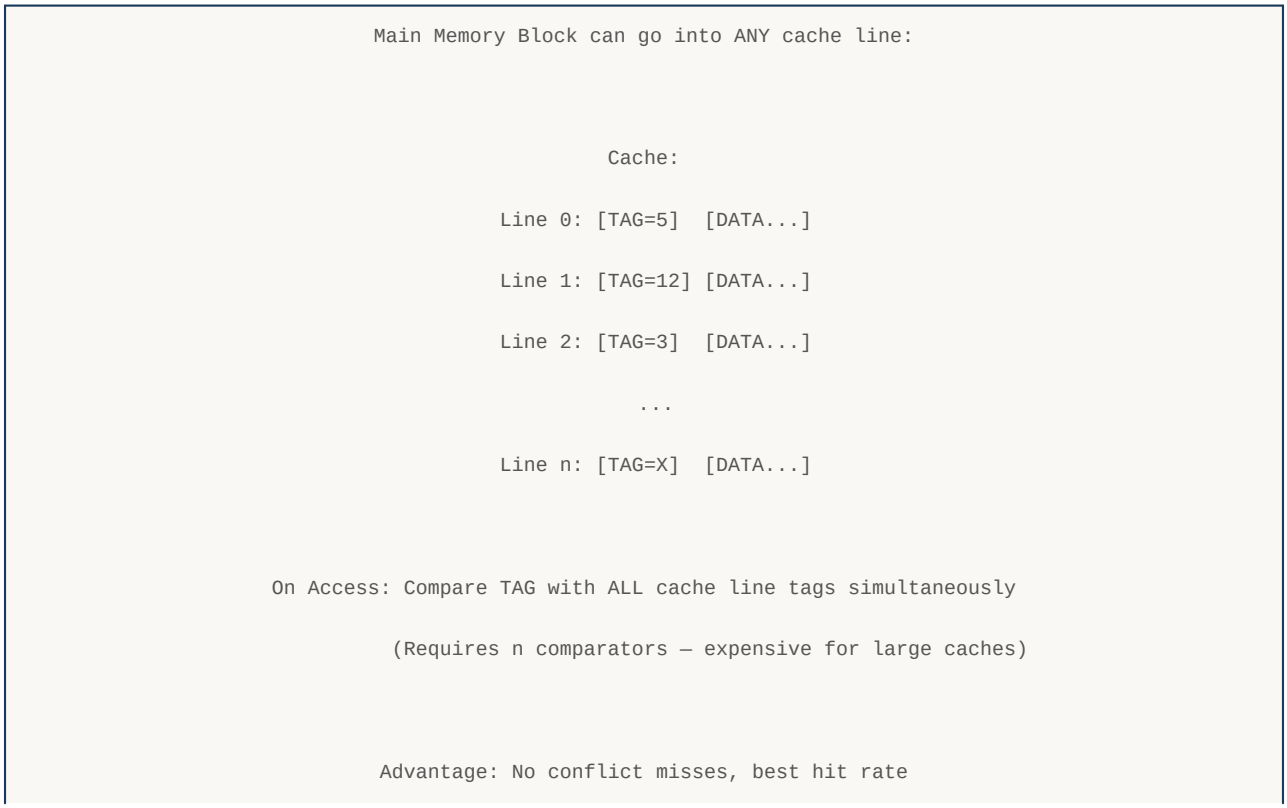


Fig 9.2: Direct Mapped Cache

B) Fully Associative Mapping:

A memory block can be placed in ANY cache line. Most flexible but requires comparing all tags simultaneously (expensive hardware).



```
Disadvantage: Complex and expensive hardware, slower lookup
```

Fig 9.3: Fully Associative Cache

C) Set-Associative Mapping:

Compromise between direct and fully associative. Cache is divided into sets; each set has k lines (k-way set associative). A memory block maps to a specific set but can go in any line within that set.

```
k-Way Set Associative (2-way shown):


Memory Address: [TAG | SET NUMBER | WORD OFFSET]

                                  |

                            +-----v-----+

                  |  Set 0    |  Line 0 | Line 1   (2 lines/set)

                          |  Set 1    |  Line 0 | Line 1

                          |  Set 2    |  Line 0 | Line 1

                            |  ...      |  ...    | ...

                          |  Set S-1  |  Line 0 | Line 1

                                +-----------+


                Block maps to Set# = Block_Number mod S

           Within the set, any line can be used (2 choices in 2-way)


             k=1: Direct mapped    k=all_lines: Fully associative

           Typical: 4-way or 8-way set associative (best balance)
```

Fig 9.4: Set-Associative Cache

Cache Replacement Policies:

Policy	Description	Advantage
LRU (Least Recently Used)	Replace line not used for longest time	Best hit rate, mimics real access patterns
FIFO (First In First Out)	Replace oldest line in cache	Simple to implement
LFU (Least Frequently Used)	Replace line with fewest accesses	Good for stable, repeated access patterns
Random	Replace a randomly selected line	Very simple, surprisingly effective
Optimal (OPT)	Replace line not needed for longest future time	Best possible — not implementable (needs future)

Q4. Virtual Memory (Paging, Segmentation, Demand Paging)

Virtual memory allows programs to use more memory than physically available by using disk as an extension of RAM. Programs see a large virtual address space regardless of actual physical memory.

Paging:

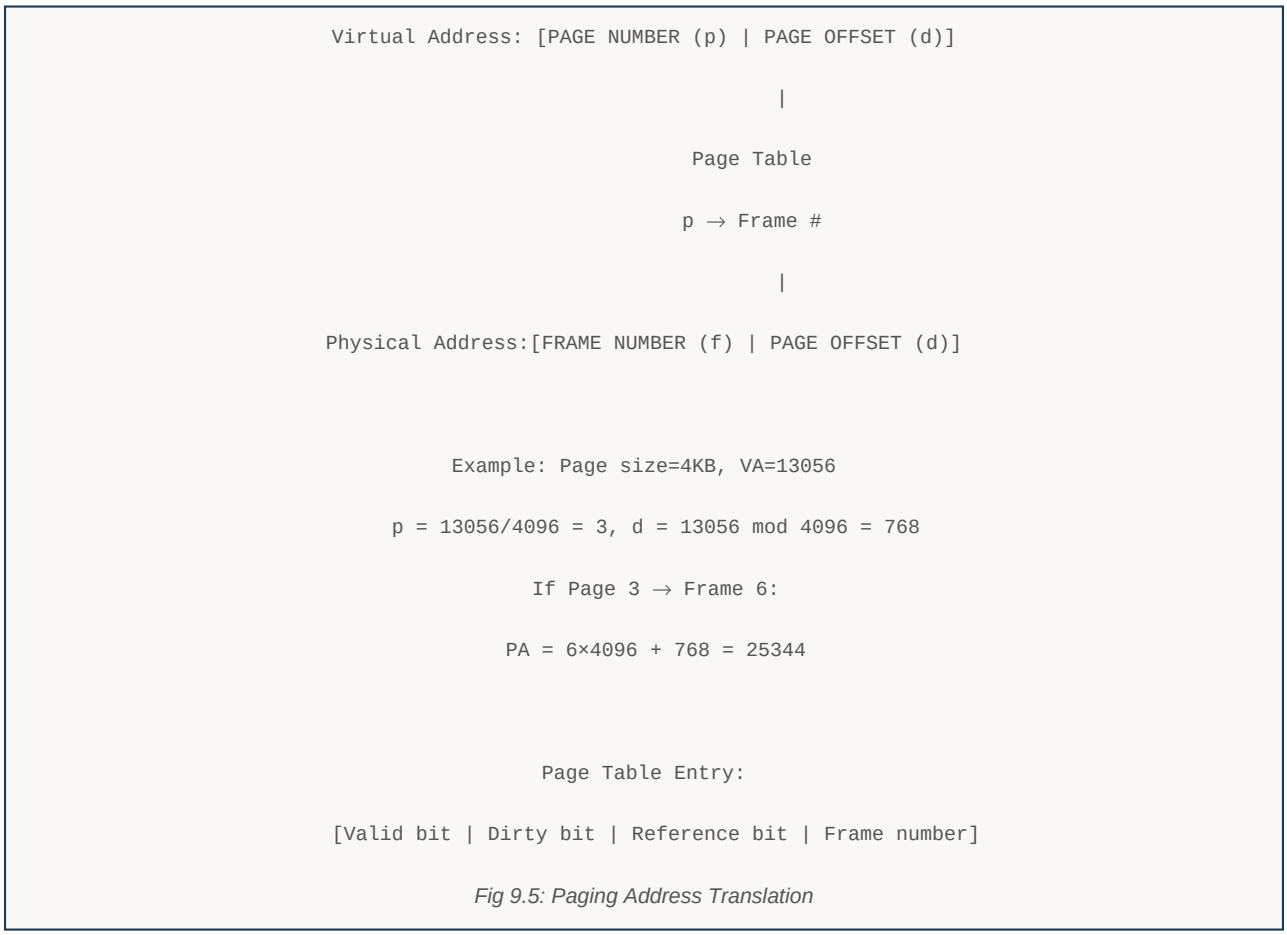


Fig 9.5: Paging Address Translation

Segmentation:

Logical address space divided into variable-size segments corresponding to program modules (code, data, stack, heap).

```
Segment Table: Segment# → (Base, Limit)

Physical Address = Base[Segment#] + Offset
                   (if Offset < Limit[Segment#])

Example:
  Segment 2 (Data): Base=4000, Limit=1000
  Logical address [2, 600] → PA = 4000 + 600 = 4600 ✓
  Logical address [2, 1200] → TRAP (exceeds limit!)
```

Demand Paging:

Pages loaded into memory only when accessed (on demand). If page not in memory, page fault occurs, OS fetches from disk.

- Valid-Invalid bit in page table: Valid=in memory, Invalid=on disk.
- Page fault: CPU generates exception, OS handles it.
- OS selects victim frame (using replacement algorithm), loads needed page.
- Advantages: Less memory needed, faster startup, more processes can run.

Q5. Associative Memory (Content Addressable Memory)

Associative memory (CAM) is searched by content rather than by address. You specify what data you're looking for, and the memory returns the address where it is stored — or the data itself.



Unit 10: Computer Arithmetic

4 Hours 10%

Q1. Addition and Subtraction (Signed Magnitude, 2's Complement)

Signed Magnitude Representation:

- MSB = sign bit (0 for positive, 1 for negative).
- Remaining bits = magnitude.
- Example (8-bit): +13 = 00001101, –13 = 10001101.
- Problem: Two representations of zero (+0 and –0).

2's Complement (Most Common):

```
To negate: invert all bits, then add 1.

  +13 in 8-bit 2's complement = 00001101
  Step 1: Invert = 11110010
  Step 2: Add 1  = 11110011 = -13 ✓

Addition Examples:
  (+7) + (+3) = 00000111 + 00000011 = 00001010 = +10 ✓
  (+7) + (-3) = 00000111 + 11111101 = 00000100 = +4 ✓  (carry discarded)
  (-7) + (-3) = 11111001 + 11111101 = 11110110 = -10 ✓ (carry discarded)

Overflow Detection:
  Overflow if carry into MSB ≠ carry out of MSB
  OR: Overflow if adding two positives gives negative
      OR adding two negatives gives positive
```

Q2. Booth Multiplication Algorithm

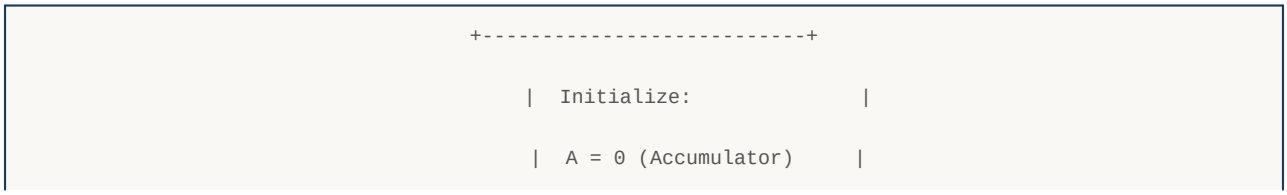
> **IMPORTANT:** Booth's Algorithm is VERY IMPORTANT for GTU — always asked with flowchart and numerical example.

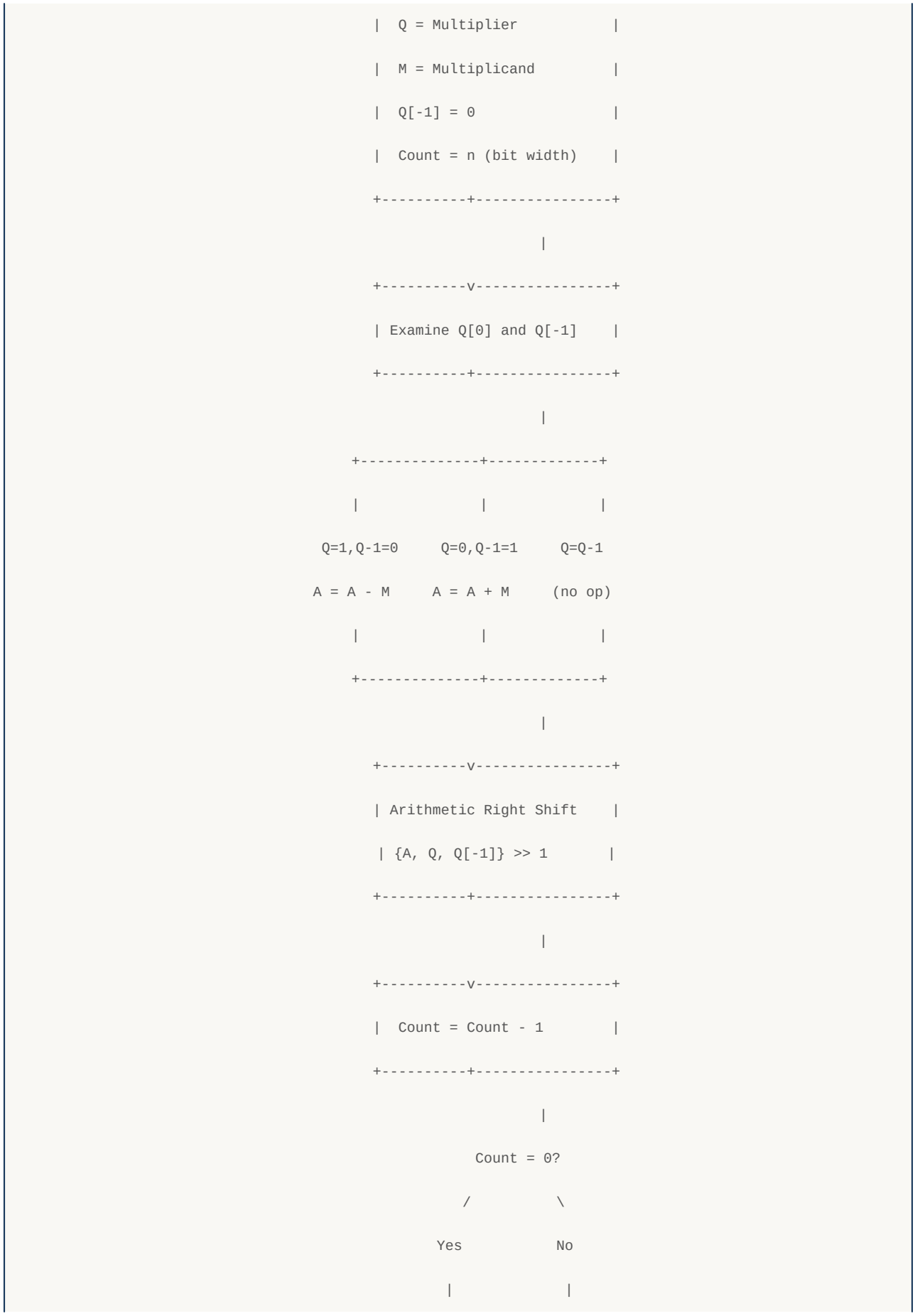
Booth's algorithm multiplies two signed binary numbers (in 2's complement) by examining pairs of bits in the multiplier. It reduces the number of operations by replacing consecutive 1s with addition and subtraction.

Booth's Algorithm Rules:

Current Bit (Qn)	Previous Bit (Qn-1)	Operation
0	0	No operation — shift right
0	1	Add Multiplicand (A ← A + M), then shift right
1	0	Subtract Multiplicand (A ← A – M), then shift right
1	1	No operation — shift right

Booth's Algorithm Flowchart:




| Q = Multiplier |
| M = Multiplicand |
| Q[-1] = 0 |
| Count = n (bit width) |
Examine Q[0] and Q[-1]
Q=1,Q-1=0
A = A - M
Q=0,Q-1=1
A = A + M
Q=Q-1
(no op)
Arithmetic Right Shift
{A, Q, Q[-1]} >> 1
Count = Count - 1
Count = 0?
Yes
No

```
                                    RESULT        (loop back)

                                  (A:Q is product)
```

Fig 10.1: Booth's Multiplication Algorithm Flowchart

Booth's Algorithm Numerical Example: 3 × (−2)

```
M (Multiplicand) = +3 = 0011  (4-bit)
Q (Multiplier)   = -2 = 1110  (4-bit, 2's complement)
-M = -3 = 1101

Initialize: A=0000, Q=1110, Q[-1]=0, Count=4

Step 1: Q[0]=0, Q[-1]=0 → No operation
        Arithmetic Right Shift: A=0000, Q=1110→0111, Q[-1]=0
        Count=3

Step 2: Q[0]=1, Q[-1]=0 → A = A - M = 0000 - 0011 = 1101
        Arithmetic Right Shift: A=1110, Q=1011, Q[-1]=1
        (shift right with sign extension: 1101→1110, Q: 1011)
        Count=2

Step 3: Q[0]=1, Q[-1]=1 → No operation
        Arithmetic Right Shift: A=1111, Q=0101, Q[-1]=1
        Count=1

Step 4: Q[0]=1, Q[-1]=1 → No operation
        Arithmetic Right Shift: A=1111, Q=1010, Q[-1]=1
        Count=0

Result: A:Q = 1111 1010 = 11111010 in 8-bit
        = -6 in decimal ✓  (3 × -2 = -6 ✓)
```

Q3. Division Algorithms

Restoring Division:

- Similar to manual long division.
- Shift Dividend left, subtract Divisor.
- If result positive: set quotient bit=1, keep result.
- If result negative: set quotient bit=0, restore (add back divisor).

```
Divide 7 by 2 (0111 / 0010):
  Dividend = 0111, Divisor = 0010

  Step 1: Shift left. A:Q = 0000 0111
  Step 2: A - Divisor = 0000 - 0010 = 1110 (negative)
           Restore: A = 0000; Q[0] = 0
  Step 3: Shift left, try again...

  After 4 steps: Quotient Q = 0011 = 3
                 Remainder A = 0001 = 1
```

```
  Check: 7 = 2×3 + 1 ✓
```

Q4. Floating Point Arithmetic Operations

Floating Point Addition/Subtraction:

```
Steps for FP Addition (A + B):
1. Compare exponents
2. Shift mantissa of smaller exponent to right (align)
3. Add/Subtract mantissas
4. Normalize result (adjust mantissa and exponent)
5. Round result to fit format

Example: 1.5 × 2^3 + 1.25 × 2^3
  Step 1: Exponents equal (both 3) — no alignment needed
  Step 2: No shift needed
  Step 3: 1.5 + 1.25 = 2.75
  Step 4: 2.75 = 1.375 × 2^1 → Normalize: 1.375 × 2^(3+1)?
          Actually: 2.75 = 1.375 × 2^1
          But aligned to exponent 3: result exponent=4, mantissa=1.375?
          Let's redo: 1.5×2^3 = 12, 1.25×2^3=10, sum=22=1.375×2^4
  Result: 1.375 × 2^4 = 22 ✓
```

Floating Point Multiplication:

```
Steps for FP Multiplication (A × B):
1. Add exponents: E_result = E_A + E_B - Bias
2. Multiply mantissas: M_result = M_A × M_B
3. Normalize result
4. Round to fit format

Example: (1.5 × 2^3) × (1.25 × 2^2)
  Exponent: 3 + 2 = 5
  Mantissa: 1.5 × 1.25 = 1.875
  Result: 1.875 × 2^5 = 60 ✓
  (Check: 1.5×2^3 = 12, 1.25×2^2 = 5, 12×5=60 ✓)
```

Q5. Decimal Arithmetic Unit

Decimal arithmetic operates on BCD (Binary Coded Decimal) where each decimal digit is represented by 4 bits (0000=0, 0001=1, ..., 1001=9).

```
BCD Addition: 4 + 7 = 11
  0100 (4)
+ 0111 (7)
= 1011 (binary) = 11 (invalid BCD — > 9)

Correction: Add 6 (0110) when result > 9 or carry occurs:
  1011 + 0110 = 10001
  = 0001 0001 = 1 1 = 11 (BCD) ✓

BCD Addition: 5 + 8 = 13
  0101 (5)
+ 1000 (8)
```

```
  = 1101 (> 9, invalid)
  + 0110 (add 6 for correction)
  = 10011 → carry=1, result=0011
  = 0001 0011 = 13 (BCD) ✓
```

Unit 11: Multiprocessors

4 Hours 5%

Q1. Characteristics of Multiprocessors

A multiprocessor system consists of two or more processors that share a common main memory and are controlled by a single operating system.

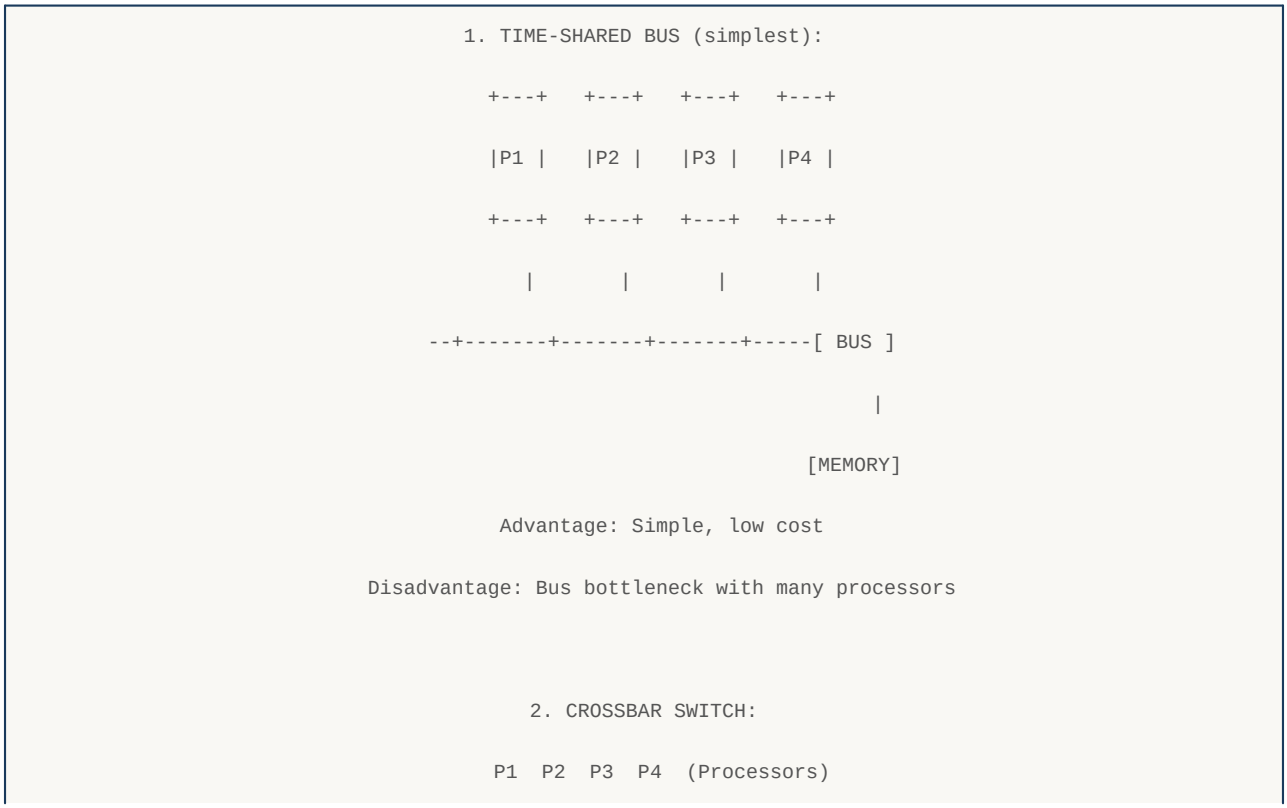
- Multiple CPUs share a common physical memory.
- Processors communicate through shared memory.
- Single operating system manages all processors.
- Processors can run different programs or cooperate on one program.
- **Tightly coupled:** Processors share memory — fast communication.
- Examples: Symmetric Multiprocessors (SMP), Multi-core CPUs.

Types:

Type	Description	Example
UMA (Uniform Memory Access)	All processors access all memory with same latency	Traditional SMP systems
NUMA (Non-Uniform Memory Access)	Each processor has local memory; faster to access local than remote	Large server systems, NUMA workstations
COMA (Cache-Only Memory)	All memory is cache; data migrates to where it is needed	Research systems

Q2. Interconnection Structures

Interconnection networks connect processors to each other and to memory modules. The choice affects performance, scalability, and cost.



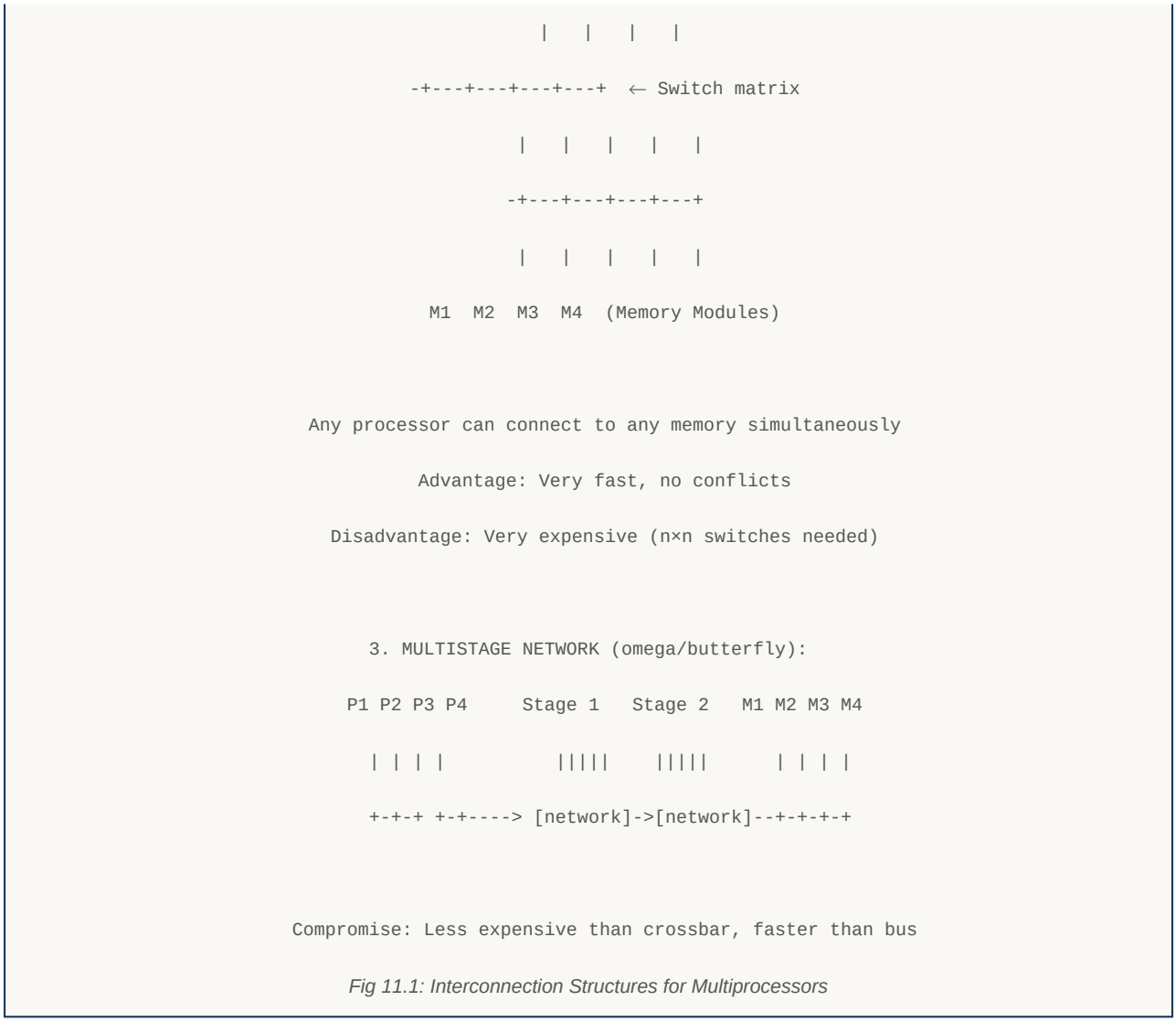


Fig 11.1: Interconnection Structures for Multiprocessors

Q3. Inter-processor Arbitration, Communication, and Synchronization

Arbitration:

When multiple processors want to access the shared bus or memory simultaneously, arbitration determines who gets access.

- **Centralized Arbitration:** A single bus arbiter grants access. Simple but can become bottleneck.
- **Distributed Arbitration:** Processors negotiate among themselves (e.g., daisy chain, self-selection).
- **Priority-based:** Higher priority processor wins.
- **Round-Robin:** Each processor takes turns — fair but not always efficient.

Communication:

- **Shared Memory:** Processors read/write shared variables. Fast but needs synchronization.
- **Message Passing:** Processors send explicit messages. Used in distributed systems.
- **Semaphores:** Used to synchronize access to shared resources.

Synchronization:

Synchronization ensures correct ordering of operations across processors.

```
Test-and-Set (hardware atomic operation):
  TS(lock):  temp = lock
```

```
              lock = 1      (atomic — cannot be interrupted)
              return temp

  Acquire lock:  while(TS(lock) == 1);  // spin until lock=0
  Release lock:  lock = 0;

Spinlock usage:
  while(TS(&mutex;) != 0);   // Busy wait for lock
    // Critical Section
  mutex = 0;                 // Release lock

Barrier Synchronization:
  All processors must reach barrier before any proceeds
  Used in parallel loop execution
```

Q4. Cache Coherence

In multiprocessor systems, each processor may have its own cache. **Cache coherence** ensures that all processors see a consistent view of memory — i.e., if one processor modifies a value, other processors must not use a stale cached copy.

The Problem:

```
P1 reads X → P1 cache: X=5, Memory: X=5
P2 reads X → P2 cache: X=5, Memory: X=5
P1 writes X=10 → P1 cache: X=10
P2 reads X → P2 cache still: X=5  ← STALE DATA! Problem!
```

Cache Coherence Protocols:

Write-Invalidate (most common — MESI protocol):

- When processor writes to a cache line, it **invalidates** all other caches holding that line.
- Other processors must fetch fresh copy from memory/cache next time they need it.
- Used by Intel processors.

Write-Update (Write-Broadcast):

- When processor writes, it broadcasts update to all other caches.
- All copies updated simultaneously.
- Higher bus traffic but no need to re-fetch.

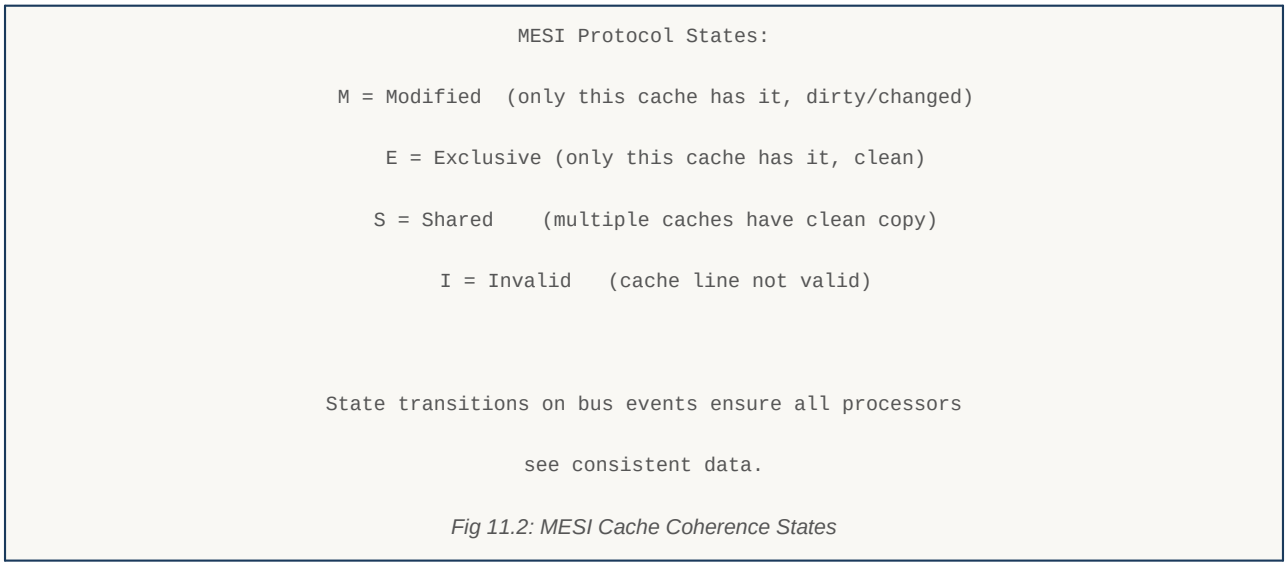
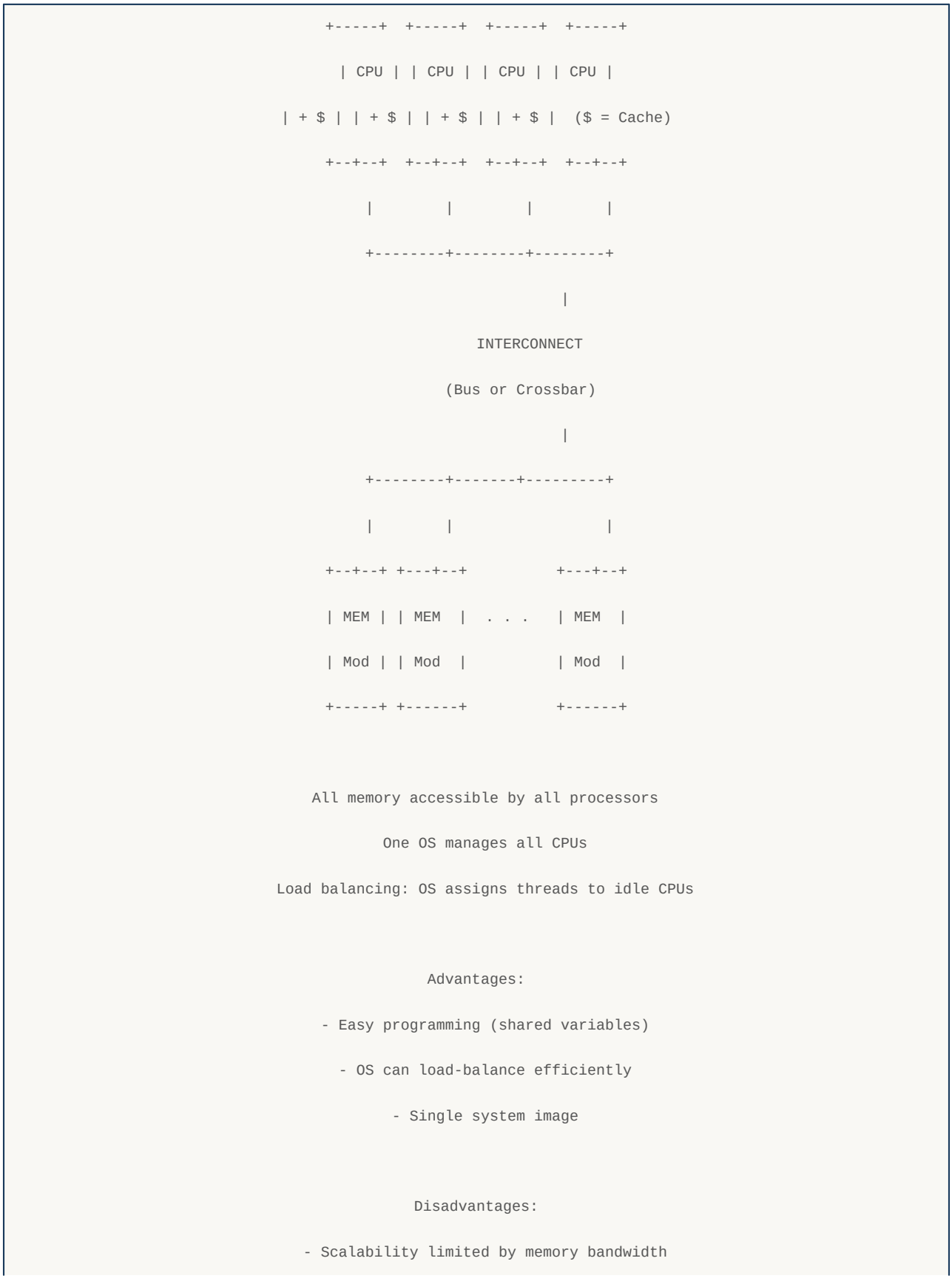


Fig 11.2: MESI Cache Coherence States

Q5. Shared Memory Multiprocessors

In shared memory multiprocessors (SMP — Symmetric Multiprocessing), all processors have equal access to all memory locations. There is one OS that sees and manages all processors.



```
- Cache coherence overhead increases with processors

- Typically limited to ~32-64 processors
```

Fig 11.3: Shared Memory Multiprocessor Architecture

GTU Exam Preparation Tips

High Priority Topics by Marks

Unit	Topic	Marks %	Must Prepare
3 & 6	Basic Computer + CPU	15%+15%=30%	Instruction cycle, RISC vs CISC, Addressing modes
10	Computer Arithmetic	10%	Booth's Algorithm (numerical + flowchart)
9	Memory Organization	10%	Cache mapping (all 3 types), Virtual memory
8	I/O Organization	10%	DMA, Interrupts, Handshaking, Strobe
7	Pipeline	10%	Pipeline stages, Hazards, Flynn's taxonomy
2	Register Transfer	10%	RTL notation, ALU, Micro-operations
4	Assembly Language	10%	Programs, Addressing modes
1	Computer Structure	5%	Von Neumann vs Harvard, IEEE 754
5	Microprogramming	5%	Hardwired vs Microprogrammed
11	Multiprocessors	5%	Cache coherence, MESI, Interconnects

Most Repeated GTU Questions

- RISC vs CISC — Comparison table (Unit 6)
- Booth's Multiplication Algorithm with example (Unit 10)
- Cache mapping techniques — Direct, Associative, Set-Associative (Unit 9)
- Instruction Cycle (Fetch-Decode-Execute) with RTL (Unit 3)
- Pipeline stages, speedup formula, and hazards (Unit 7)
- DMA vs Interrupt-driven vs Programmed I/O (Unit 8)
- Von Neumann vs Harvard architecture (Unit 1)
- IEEE 754 floating point representation with example (Unit 1)
- Flynn's Taxonomy — SISD, SIMD, MISD, MIMD (Unit 7)
- Addressing modes with examples — all types (Unit 4 & 6)
- Hamming code error correction (Unit 1)
- Cache coherence and MESI protocol (Unit 11)

Reference Books

- Morris Mano — Computer System Architecture (Primary textbook for GTU)
- William Stallings — Computer Organization and Architecture
- Patterson & Hennessy — Computer Organization and Design (RISC-V Edition)
- Hamacher, Vranesic, Zaky — Computer Organization